

LitFolio 用户手册

本地优先的研究文献工作台

v0.3 · 2026 年 5 月

LitFolio 项目组

本手册由 L^AT_EX 排版生成。

目录

前言	vii
第一部分 日常工作流	1
第一章 起步	2
1.1 什么是 LitFolio	2
1.2 安装与首次启动	2
1.2.1 从发布版下载（推荐）	2
1.2.2 从源码运行（开发者）	3
1.3 库目录第一眼	3
1.4 下一步：配第一个 LLM 端点	4
第二章 文献库	5
2.1 主列表与搜索	5
2.1.1 搜索框	5
2.1.2 FTS5 全文检索机制	5
2.1.3 排序与筛选	6
2.2 阅读状态：unread → reading → read → must	6
2.3 速读 TL;DR	6
2.4 深读：4 段式抽屉	6
2.5 标题/摘要翻译	6
2.6 文件夹	7
2.7 标签	7
2.8 删除论文	7
2.9 BibTeX 自动生成	8
2.9.1 BibTeX Key 生成算法	8
2.9.2 条目字段映射	9
2.9.3 入库时自动生成	10
2.10 引用格式化	10
2.10.1 四种样式模板	10
2.10.2 批量格式化引用	11
2.11 批量引用导出	11

2.11.1 BibTeX (.bib)	11
2.11.2 RIS (.ris)	11
2.11.3 格式化文本	12
2.11.4 文件命名与触发	12
2.12 智能收藏夹	12
2.12.1 规则结构	12
2.12.2 SQL 翻译引擎	13
2.12.3 实时更新	14
2.12.4 性能	14
2.13 阅读队列	14
2.13.1 队列数据模型	15
2.13.2 拖拽排序机制	15
2.13.3 目标日期与超期提醒	15
2.13.4 自动状态更新	15
2.13.5 队列与文件夹/标签的关系	15
2.14 论文去重	16
2.14.1 三级匹配策略	16
2.14.2 标题预处理	16
2.14.3 全库扫描	16
2.14.4 合并流程	17
第三章 导入文献	18
3.1 arXiv ID / DOI	18
3.2 本地 PDF 文件	19
3.2.1 PDF 元数据提取策略	19
3.3 Semantic Scholar 搜索	20
3.4 批量文件夹导入	20
3.4.1 递归扫描	20
3.4.2 文件名启发式解析	21
3.4.3 并发处理与进度追踪	21
3.5 从 RSS / arXiv 浏览跳转	21
第四章 阅读器	22
4.1 三栏布局	22
4.1.1 PDF 全文提取与缓存	22
4.2 高亮: 文本选区 + 区域	23
4.3 选段翻译	23
4.4 术语表与术语网络	24
4.4.1 术语提取算法	24
4.5 结构化笔记卡片	26

4.5.1 默认分区	26
4.5.2 AI 自动填充机制	27
4.5.3 分区与全文搜索	27
4.6 标注颜色语义	27
4.6.1 workflow	28
4.6.2 自定义标签	28
4.7 Markdown 笔记	28
4.8 PDF 搜索	28
4.9 页面快捷键	28
第二部分 发现与综合	30
第五章 arXiv 浏览	31
5.1 分类树	31
5.2 翻页与「已穷尽」	32
5.3 入库	32
第六章 RSS 订阅	33
6.1 订阅源管理	33
6.2 条件 GET 刷新	34
6.3 条目详情与翻译	34
6.4 从 feed item 直接入库	34
第七章 主题发现	35
7.1 两 tab 心智模型	35
7.2 搜索召回	35
7.2.1 扩写查询	35
7.2.2 扩写算法细节	35
7.2.3 时间窗与排序	36
7.3 综述生成	36
7.3.1 流水线	36
7.3.2 保存与恢复	38
7.4 主题提醒	38
7.4.1 创建提醒	39
7.4.2 提醒执行流程	39
7.4.3 自动入库	40
7.4.4 管理与查看	40

第八章 库内提问 (RAG)	41
8.1 工作机制	41
8.2 多轮对话界面	42
8.2.1 对话状态模型	42
8.2.2 多轮 RAG 流程	43
8.2.3 对话历史传递的细节	43
8.2.4 UI 交互	43
8.3 保存为知识笔记	44
第九章 知识图谱	45
9.1 网络图	45
9.2 思维导图	46
9.3 手动创建关系	46
9.4 引用网络	47
9.4.1 数据获取流程	47
9.4.2 树形展示	48
9.4.3 与知识图谱的关系	48
9.5 概念图谱	48
9.5.1 概念提取流水线	48
9.5.2 概念关系类型	49
9.5.3 图谱可视化集成	50
9.5.4 手动管理	50
9.6 AI 发现关系	50
9.7 筛选与图例	51
9.8 阅读器集成	51
第十章 相似论文推荐	52
10.1 使用方式	52
10.1.1 推荐获取流程	52
10.1.2 结果展示	53
10.1.3 边界情况	53
第十一章 文献综述草稿生成	54
11.1 使用方式	54
11.1.1 生成流水线	54
11.1.2 输入材料	55
11.2 输出与编辑	56

第三部分 高级工具与运维	57
第十二章 多论文对比	58
12.1 使用方式	58
12.1.1 对比生成流程	58
12.1.2 对比表格结构	59
12.1.3 编辑与保存	59
第十三章 Markdown 导出	60
13.1 导出内容	60
13.1.1 文件结构	60
13.1.2 Frontmatter 字段	61
13.1.3 高亮按标签分组	62
13.1.4 术语与双向链接	62
13.2 增量导出算法	62
13.3 配置与触发	62
第十四章 命令面板	63
14.1 三种模式	63
14.1.1 模糊匹配算法	64
14.1.2 结果分组与展示	64
第十五章 自定义元数据字段	65
15.1 字段管理	65
15.1.1 字段类型	65
15.1.2 数据模型	65
15.2 使用	65
第十六章 设置	67
16.1 LLM 配置	67
16.1.1 拉取可用模型	68
16.1.2 测试连接	68
16.2 任务分配	68
16.2.1 各任务的 LLM 调用细节	68
16.3 WebDAV 同步	69
16.4 导出设置	70
16.5 标注颜色设置	70
16.6 自定义字段设置	70
16.7 主题提醒设置	70

第十七章 数据与文件布局	72
17.1 顶层结构	72
17.2 library.db 表概览	72
17.2.1 核心	72
17.2.2 发现与阅读	73
17.2.3 笔记与知识	73
17.2.4 配置与索引	73
17.3 papers/<id>/ 单篇布局	74
17.4 性能优化	74
17.4.1 虚拟滚动	74
17.4.2 嵌入缓存	74
17.5 备份	75
第十八章 快捷键速查	76
18.1 阅读器 (/reader)	76
18.2 全局	76
第十九章 故障排查	77
19.1 LLM 调用失败	77
19.2 PDF 加载失败	77
19.3 RSS 抓不到 / 一直「已是最新」	77
19.4 WebDAV 同步失败	78
19.5 数据库锁住	78
19.6 批量导入部分失败	78
19.7 概念提取结果为空	78
19.8 引用网络/相似论文为空	79
19.9 智能收藏夹不更新	79
附录 A LLM 端点兼容清单	80
附录 B arXiv 主流分类速查	81
附录 C 推荐 RSS 源	82
附录 D 许可证	83

前言

这本手册是什么

LitFolio 是一款本地优先 (local-first) 的桌面研究文献工作台。它把 arXiv 浏览、PDF 阅读、高亮笔记、术语提取、多轮 RAG 对话、知识图谱、主题综述生成、文献对比、引用管理、WebDAV 同步全部装进一个 Tauri 应用里，所有数据都落在你自己电脑上的 `~/Litera-Library/` 目录。

这本手册写给两类读者：

- **新用户**——从未用过 LitFolio，希望在半小时内能把一篇 arXiv 论文从导入到深读、翻译、术语网络走通。看第 1 至 4 章即可。
- **已经在用的研究者**——想吃透「主题综述」、多轮 RAG 对话、知识图谱、WebDAV 同步等进阶功能，并了解数据是怎么落盘的。看 Part II、Part III 与第 12 章。
- **重度用户**——需要文献对比、综述草稿、BibTeX 管理、智能收藏夹、命令面板等效率工具。重点看 Part III（第 11–17 章）。

你将学到什么

1. 三种文献导入方式 (arXiv/DOI、PDF 文件、Semantic Scholar 搜索)，批量文件夹导入，以及自动去重检测。
2. 文献阅读的全流程：列表 → 速读 TL;DR → 深读四段式 → 翻译 → 打开 PDF 阅读器 → 高亮（语义标注颜色）→ 结构化笔记卡片 → 术语网络。
3. BibTeX 自动生成与引用格式化 (APA/IEEE/GB/T 7714)，批量导出为 .bib 或 .ris 文件。
4. 把若干篇散乱的论文整合成主题综述，或就「这些论文里讨论 CPA 局限的工作有哪几篇？」这类问题让 LitFolio 直接给出带证据编号的答案。支持多轮追问对话。
5. 知识图谱：论文关系网络 + 引用网络 + 概念图谱，AI 自动发现方法演化与跨论文关联。
6. 进阶工具：多论文结构化对比、文献综述草稿生成、相似论文推荐、阅读队列、智能收藏夹、自定义元数据字段、Markdown/Obsidian 导出、命令面板 (`Ctrl+K`)。
7. 配置 LLM 端点、把不同任务分派给不同模型；通过 WebDAV 把整个库同步到另一台机器。
8. 数据落盘的物理结构，便于你做备份、写脚本或排查故障。

阅读约定

蓝紫色 章节标题与强调，对应产品中的主交互色（DESIGN token violet）。

Ctrl+F 快捷键以圆角芯片样式呈现。

file.tsx 文件路径与代码标识符使用等宽字体。

pnpm tauri dev 终端命令同样使用等宽字体。

下面这两类提示框会在本手册中反复出现：

提示 浅紫色框：补充说明、加速使用的小技巧，可跳过。

注意 琥珀色框：操作不可逆 / 会覆盖数据 / 需要先理解再执行，请认真读一遍。

反馈与协助

LitFolio 是 GPL-3.0-or-later 开源软件。问题反馈、功能建议、补丁请到项目仓库的 **issue tracker** 提交。本手册随主仓库同源，欢迎一并提 **PR** 改正错别字或补充用例。

第一部分

日常 workflow

1.1 什么是 LitFolio

LitFolio 是一款桌面端的研究文献工作台，核心理念是**本地优先 (local-first)**：所有数据——元数据、PDF、笔记、高亮、术语、综述——都直接落在你机器的 `~/Litera-Library/` 下；联网只用来抓 arXiv 元数据、解析 RSS、请求 LLM。换言之，断网你照样能读、能写、能搜索；换台机器只需要把这个目录同步过去。

LitFolio 的产品形态由八大功能板块拼起来：

1. **文献库**——本地论文的列表、状态机、TL;DR、深读、翻译、文件夹、标签、BibTeX 自动生成、引用格式化、批量导出。
2. **阅读器**——三栏 PDF 视图，左侧高亮列表、右侧笔记/选段翻译/术语三 tab；结构化笔记卡片、语义标注颜色。
3. **导入**——arXiv/DOI 元数据、本地 PDF 文件、Semantic Scholar 搜索三种入口；批量文件夹导入与去重检测。
4. **发现**——arXiv 分类浏览 + RSS 订阅 + 主题综述生成 + 主题提醒监控。
5. **提问**——基于已读论文的多轮 RAG 对话：问题改写、多路 FTS5 召回、带引用编号的回答、上下文追问。
6. **知识图谱**——论文关系网络 + 引用网络 + 概念图谱，AI 自动发现关联与方法演化。
7. **高级工具**——多论文对比、文献综述草稿、相似论文推荐、阅读队列、智能收藏夹、自定义元数据字段、Markdown 导出、命令面板。
8. **设置**——多个 LLM 端点配置、把不同任务分派给不同模型、WebDAV 整库同步。

1.2 安装与首次启动

LitFolio 用 Tauri 2 + React 18 构建，提供两种获取方式：

1.2.1 从发布版下载（推荐）

到项目仓库的 Releases 页面下载对应平台的安装包：

- Linux: [LitFolio_x.y.z_amd64.AppImage](#) 或 [.deb](#)
- macOS: [LitFolio_x.y.z_universal.dmg](#)
- Windows: [LitFolio_x.y.z_x64-setup.exe](#)

1.2.2 从源码运行（开发者）

1. 装好 pnpm、rustup（含 stable toolchain）以及对应平台的 Tauri 系统依赖（Linux 上一般是 webkit2gtk-4.1、librsvg、libsoup-3.0）。
2. pnpm install 安装前端依赖。
3. pnpm tauri dev 启动开发模式；首次启动会调好 Rust crate，稍等几分钟。

提示 开发模式下窗口标题会带上「[dev]」字样；与生产版数据库相同（都落在 [~/Litera-Library/](#)），所以在 dev 里做的导入、深读、笔记，切到打包版同样能看见。

1.3 库目录第一眼

第一次启动后，LitFolio 会自动创建 [~/Litera-Library/](#) 并初始化空数据库。你可以立刻看到左侧导航栏（[文献库](#) / [导入](#) / [浏览](#) / [订阅](#) / [主题](#) / [提问](#) / [设置](#)）和空空如也的主列表。

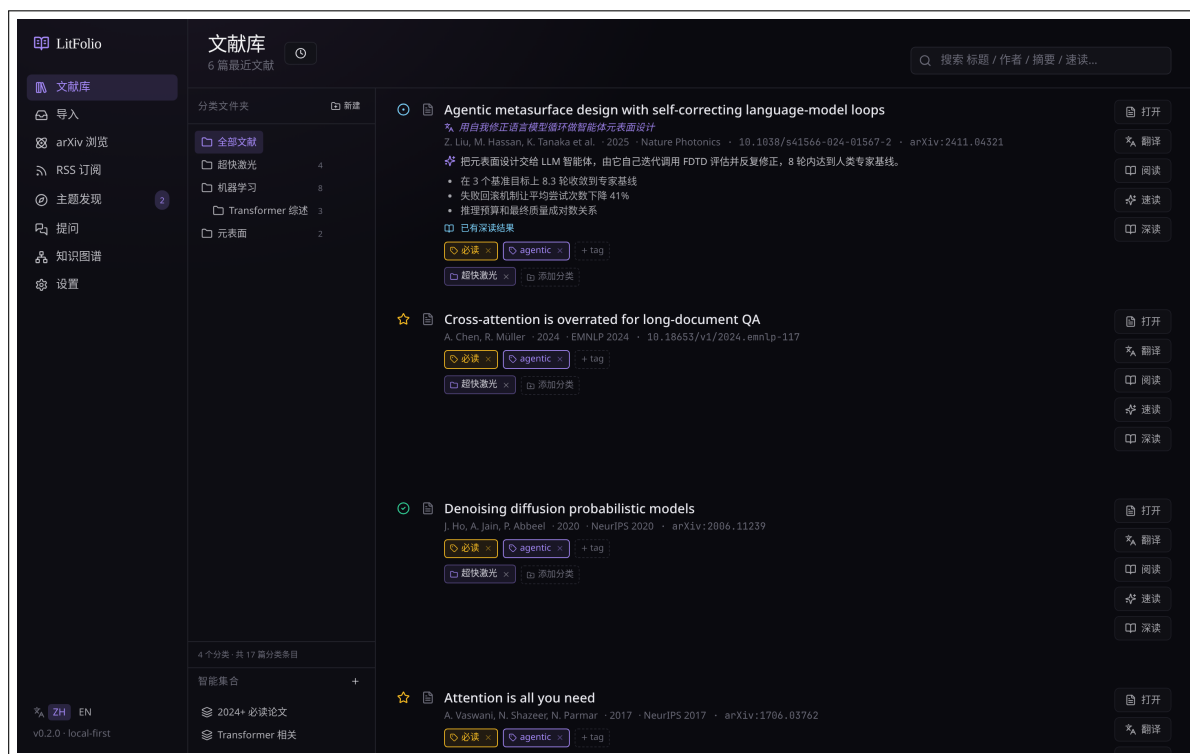


图 1.1 LitFolio 主界面。左：导航栏；中：文件夹侧栏；右：论文列表与搜索框。深紫色 violet 是主交互色，用于按钮、链接和强调。

库目录的物理结构（详见第 10 章）大致是：

```
~/Litera-Library/  
library.db           // SQLite 主库  
papers/<paperId>/    // 每篇一目录  
paper.pdf  
meta.json  
note.md  
pdf-text.txt  
backups/             // 自动备份  
sync/                // 同步快照
```

1.4 下一步：配第一个 LLM 端点

LitFolio 自身不内置任何 LLM。要使用 TL;DR、深读、翻译、术语提取、主题综述、库内提问这些 AI 能力，你需要先在「设置 → LLM 配置」里添加至少一个端点（OpenAI 兼容 API 或本地 Ollama 都行）。详细配置见 **第 9 章**；你可以现在跳过去配好，再回到这里继续；也可以先用第 2 章里不依赖 LLM 的功能熟悉界面。

文献库

文献库 (/library 路由) 是 LitFolio 最常驻的页面。它的设计目标是：让你扫一眼列表就知道这堆论文都在讲什么、哪些值得深读，哪些只是 RSS 推过来还没碰过。

2.1 主列表与搜索

主列表每一行展示：标题、作者、年份/期刊、状态徽章、TL;DR 摘要（如已生成）、关键发现 chip、标签 chip。点行任意位置打开右侧抽屉（详见 2.4）。

2.1.1 搜索框

顶部搜索框走的是 SQLite FTS5：标题、摘要、TL;DR、笔记、高亮全文统一索引。关键词支持空格分词（AND 语义）。如果你想搜「正好这几个字」请用英文引号包起来。

2.1.2 FTS5 全文检索机制

LitFolio 的全文索引基于 SQLite FTS5 虚拟表 papers_fts，使用 unicode61 分词器。该表索引论文的五个字段：标题、作者、摘要、TL;DR、笔记。通过 INSERT / UPDATE / DELETE 触发器与 papers 主表保持同步——论文入库或更新时自动重索引。

搜索流程：

1. **输入清洗**——escape_fts 函数把用户输入按空格拆词，去掉 FTS5 特殊字符 ("():.-)，然后把每个词包装成 " 词"*（前缀匹配）并用 AND 连接。例如输入 retrieval augmented 变成 "retrieval"* AND "augmented"*。
2. **BM25 排序**——FTS5 内置的 BM25 算法自动计算相关度得分，结果按 BM25 升序（越小越相关）排列，同分再按入库时间倒序。
3. **空查询回退**——如果清洗后为空（比如用户只输入了特殊字符），回退到 list_recent 按时间倒序列出。

提示 FTS5 的 unicode61 分词器对中文按字切分，所以搜「神经网络」等于搜「神 AND 经 AND 网 AND 络」——短词可能召回噪声。如果需要精准中文搜索，建议用英文关键词或术语的英文名。

2.1.3 排序与筛选

侧栏的「必读/在读/已读/未读」分别对应 `must/reading/read/unread` 四个状态。点击文件夹（见 2.6）或标签 `chip` 都会做相应筛选。

2.2 阅读状态：unread → reading → read → must

每篇论文有一个 **阅读状态**，初始 `unread`：

unread 还没碰过。RSS / arXiv 入库的论文默认是这个状态。

reading 在读。表示你正在跟，希望它在列表里靠前。

read 读完了，可以从主列表淡出。

must 「必读」——明显比 `read` 更高优先级的标记，主题综述生成时也会优先纳入。

点状态徽章会按顺序循环切换。这套四态机制是 LitFolio 的一个判断：它不假设你看完一篇文献就万事大吉，而是希望保留「重要的还要常翻」这个语义。

2.3 速读 TL;DR

主列表右侧有「速读」按钮。点一下会调用「TL;DR 生成」任务（默认绑你设置的 `tldr` 模型）；返回的一段话作为论文摘要的缩写直接写入数据库，下次打开瞬间就有。

提示 速读结果一旦生成会缓存到 SQLite，再次点击同一论文不会重新调 LLM；要强制重生成请在抽屉里点「重新生成」。

2.4 深读：4 段式抽屉

「深读」是 LitFolio 区别于其它 `reader` 工具的核心交互之一。点论文行的「深读」按钮，右侧抽屉会以四段结构给出可裁的内容：

四段式的设计意图是让你不必读全文也能判断「这篇是否值得读细节」：看 **差异** 与 **局限** 两段足以决定要不要打开 PDF。整段结果同样会缓存。

2.5 标题/摘要翻译

抽屉里有「翻译」按钮，调用「翻译」任务（绑 `translate` 模型）。结果以双语对照呈现，元数据中保存翻译后的标题和摘要，方便你扫中文标题。

提示 不同任务可以绑不同模型——比如把 TL;DR 绑给 DeepSeek、把翻译绑给本地 Ollama。详第 9 章。

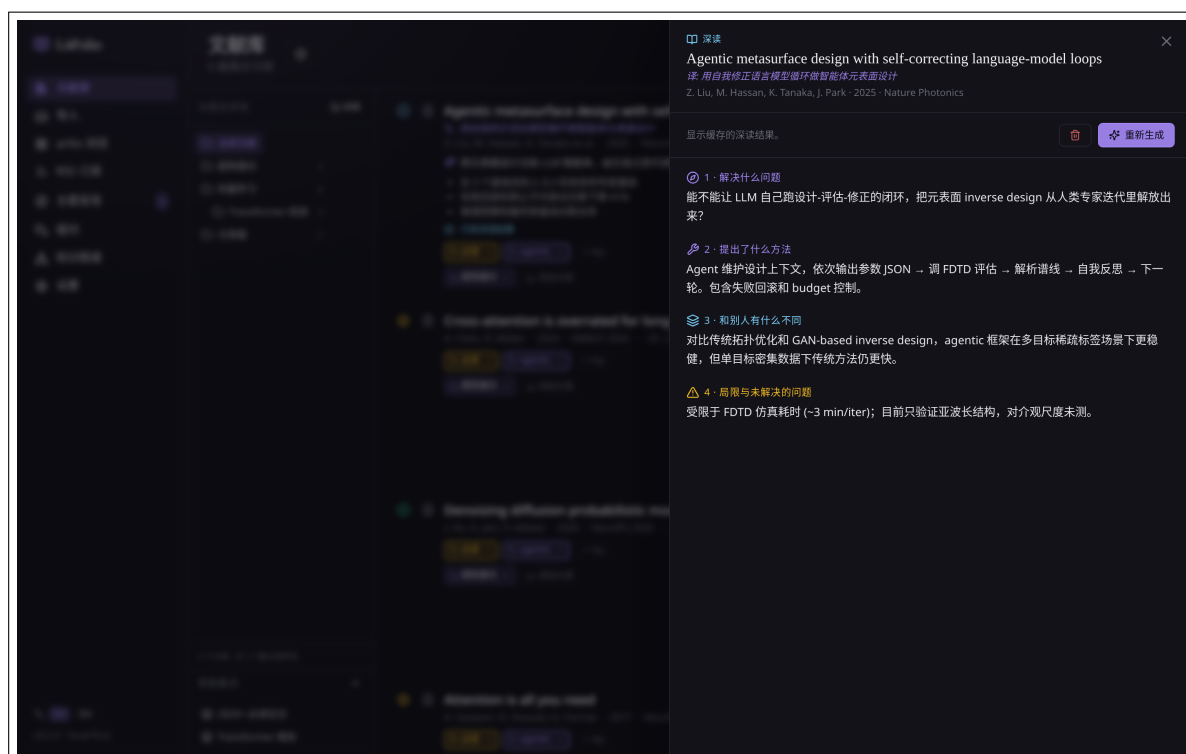


图 2.1 深读抽屉。从上到下：解决什么问题、提出了什么方法、和别人有什么不同、局限与未解决的问题——每一段都从全文中抽取，旁边有 重新生成 按钮。

2.6 文件夹

文件夹是嵌套的(不像标签),并支持多归属——一篇论文可以同时「机器学习/Transformer」和「应用/NLP」两个文件夹里。

新建文件夹：右键侧栏空白处 → 新建。论文归属管理：拖拽，或在抽屉里点「移动到…」。

2.7 标签

标签是扁平的、字符串型的，靠抽屉里的输入框创建。同样支持多挂——一篇文献可有任意多个标签。与文件夹的区别：

- 文件夹 = 研究领域分类，相对稳定，嵌套深 2-3 层。
- 标签 = 临时主题/项目标记，朝生暮死，扁平。

2.8 删除论文

抽屉底部有「删除」按钮。会同时删除：papers/<id>/ 整目录、SQLite 行、所有关联的高亮/笔记/术语。

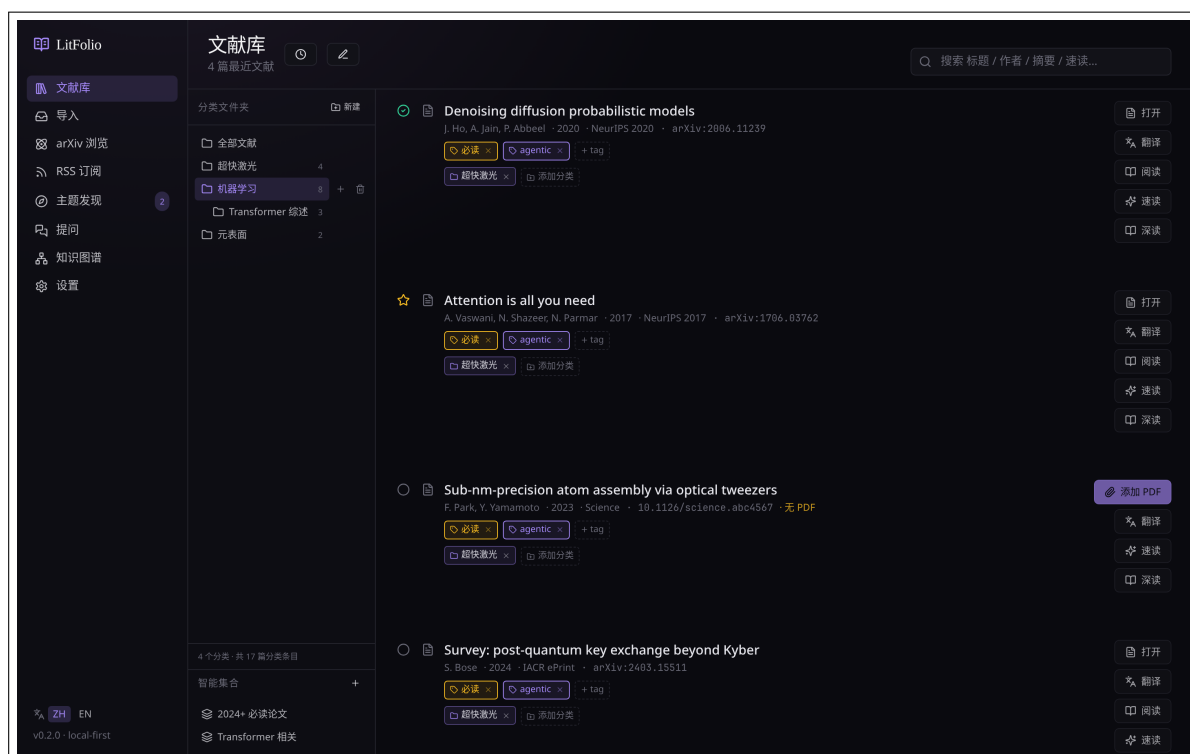


图 2.2 侧栏文件夹。选中某个文件夹后，主列表会筛出该文件夹下的论文；配合搜索框可做「文件夹内全文搜索」。

注意 删除是不可逆的,但 LitFolio 在执行前会把 `papers/<id>/` 移动到 `backups/deleted/<timestamp>/` 而不是直接 `rm -rf`。所以误删了可以从 `backups/deleted/` 里捞回。

2.9 BibTeX 自动生成

每篇论文入库时，LitFolio 会自动根据元数据生成 BibTeX 条目，存入数据库 `bibtex` 字段。抽屉和阅读器顶部都有「复制 BibTeX」按钮，点击即复制到系统剪贴板。

2.9.1 BibTeX Key 生成算法

BibTeX key 是论文的唯一引用标识符，格式为 `< 第一作者姓氏小写 >< 年份 >< 标题首实义词小写 >`，例如 `vaswani2017attention`。生成流程：

- 提取第一作者姓氏**——从 `authors` 数组取第一个元素，按空格/逗号拆分取最后一个 token（适配 Firstname Lastname 和 Lastname, Firstname 两种写法），转小写，去除非 ASCII 字符（保留 a-z 和连字符）。如果作者列表为空，使用 `unknown`。
- 提取标题首实义词**——将标题按空格拆分,跳过停用词(a/an/the/on/in/of/for/and/or/to/with/from/by/at)，取第一个非停用词，转小写，只保留 a-z0-9。如果标题全由停用词组成（极罕见），取第一个词。

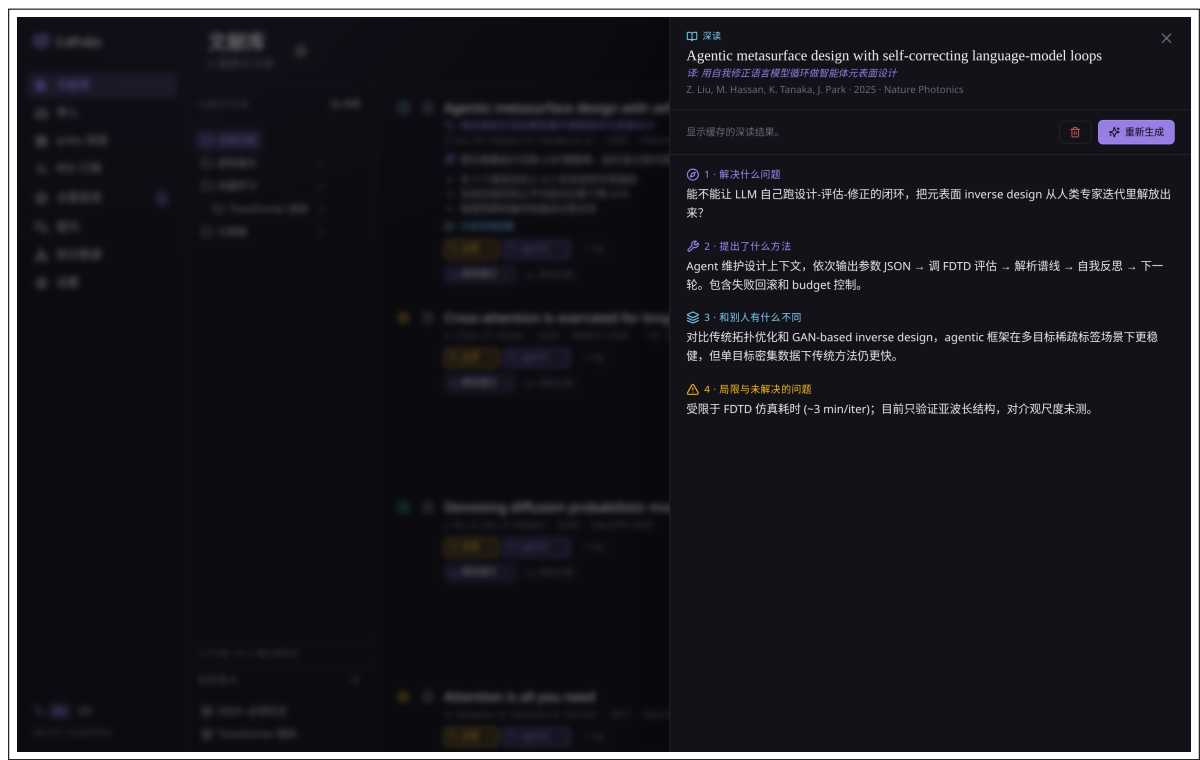


图 2.3 论文抽屉中的 BibTeX 区域。一键复制到剪贴板，可直接粘贴到 LaTeX 文档。

- 3. 拼接——< 姓氏 >< 年份 >< 首词 >。如果年份缺失，用 0000。
- 4. 去重——生成后检查库中是否已有相同 key。如果有，在末尾追加 a、b...直到唯一。这一步保证你从不同来源导入同一篇论文时 BibTeX key 不会冲突。

2.9.2 条目字段映射

BibTeX 字段	来源	处理规则
title	papers.title	直接使用
author	papers.authors	JSON 数组用 and 连接
year	papers.year	直接使用；缺失则省略
journal/booktitle	papers.venue	有 venue 时写入；缺失则省略
doi	papers.doi	有 DOI 时写入；缺失则省略
eprint	papers.arxiv_id	有 arXiv ID 时写入
archivePrefix	硬编码	arXiv ID 存在时固定为 arXiv
primaryClass	arXiv ID 解析	从 arXiv ID 前缀推断（如 cs → cs.AI）；无法推断则省略

BibTeX 字段	来源	处理规则
url	组合	有 DOI → https://doi.org/<doi>; 有 arXiv ID → https://arxiv.org/abs/<id>; 否则省略

2.9.3 入库时自动生成

BibTeX 生成发生在以下入库路径的最后一步：arXiv ID 入库、DOI 入库、PDF 文件入库、Semantic Scholar 搜索入库。生成是纯内存字符串拼接——不调 LLM、不联网、不依赖外部服务，耗时可忽略。旧论文（BibTeX 字段为 NULL）可通过设置页的「回填 BibTeX」按钮一次性补全。

提示 BibTeX key 的唯一性由数据库保证。如果你用 Zotero 或其他工具生成了不同格式的 key，LitFolio 的 key 不会与之冲突——因为 LitFolio 的 key 总是以小写姓氏开头且不含下划线。生成是纯内存字符串拼接，耗时可忽略。

2.10 引用格式化

除了 BibTeX，LitFolio 还支持按标准学术格式生成格式化引用。在论文抽屉中点「复制引用」，可选以下四种样式。格式化是纯模板渲染——将元数据字段代入预定义的字符串模板，不调 LLM。

2.10.1 四种样式模板

APA 7th

模板：< 姓 >, < 名首字母 >. (< 年 >). < 标题 >. < 期刊 >.

渲染结果示例：

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., & Polosukhin, I. (2017). Attention is all you need. *NeurIPS*.

规则：作者超过 20 人时列前 19 人 + ... + 最后一人；标题仅首词首字母大写（sentence case）；斜体期刊名。

IEEE

模板：[N] < 名首字母 >. < 姓 > et al., ``< 标题 >,' ' < 期刊 >, < 年 >.

渲染结果示例：

[1] A. Vaswani et al., “Attention is all you need,” *NeurIPS*, 2017.

规则：IEEE 用方括号编号；3 人以上用 *et al.*；标题用 Title Case；期刊斜体。

GB/T 7714-2015

模板：[N] < 姓大写 > < 名首字母大写 >, < 姓大写 > < 名首字母大写 >, ... < 标题 >[J].
< 期刊 >, < 年 >.

渲染结果示例：

[1] VASWANI A, SHAZEER N, PARMAR N, et al. Attention is all you need[J]. *NeurIPS*, 2017.

规则：中国国标格式——作者姓全大写在前、名缩写在后；超过 3 人用 *et al.*；文献类型标识 [J] 表示期刊、[C] 表示会议、[D] 表示学位论文。LitFolio 根据 venue 是否包含 Conference/Proceedings/Workshop 自动选择 [J] 或 [C]。

Chicago 17th

模板：< 姓 >, < 名 >. < 年 >. ``< 标题 >.'' < 期刊 >.

渲染结果示例：

Vaswani, Ashish, Noam Shazeer, Niki Parmar, et al. 2017. "Attention Is All You Need." *NeurIPS*.

规则：作者用全名（非缩写）；7 人以上列前 7 人 + *et al.*。

2.10.2 批量格式化引用

多选论文后点「导出引用 → 格式化文本」，LitFolio 按选定样式生成带编号的参考文献列表。编号按在列表中的出现顺序自动分配（IEEE 和 GB/T 7714 需要编号，APA 和 Chicago 按作者-年份排序）。

2.11 批量引用导出

在文献库中多选论文（勾选模式），工具栏会出现「导出引用」按钮。支持三种格式：

2.11.1 BibTeX (.bib)

将所有选中论文的 BibTeX 条目合并为一个 .bib 文件。每条记录直接取 papers.bibtex 字段（由 2.9 节的算法自动生成），用空行分隔。如果某篇论文的 BibTeX 字段为 NULL（极老的数据），跳过该条并在导出报告中列出。输出文件可直接用于 bibtex / biber / biblatex 编译。

2.11.2 RIS (.ris)

RIS（Research Information Systems）是通用文献交换格式。每条记录由标签行组成：

TY - JOUR
TI - Attention Is All You Need
AU - Vaswani, Ashish
AU - Shazeer, Noam
PY - 2017
JO - NeurIPS
DO - 10.48550/arXiv.1706.03762
ER -

标签映射规则：TY 根据 venue 判断（期刊 → JOUR，会议 → CONF，预印本 → UNPB）；AU 每个作者一行，格式为 Lastname, Firstname；DO 取 DOI；UR 取 URL。RIS 文件可直接导入 Zotero、Mendeley、EndNote。

2.11.3 格式化文本

按 2.10 节的四种样式（APA/IEEE/GB/T 7714/Chicago）生成编号引用列表。每篇论文一行或多行（取决于样式要求），编号按在列表中的出现顺序自动分配。

2.11.4 文件命名与触发

导出文件命名为 litera-export-⟨YYYY-MM-DD⟩.<ext>，通过浏览器下载对话框保存到你选择的位置。也可以选择「复制到剪贴板」直接粘贴到编辑器中。

2.12 智能收藏夹

智能收藏夹是基于规则的虚拟文件夹——论文按规则自动归入，无需手动拖拽。侧栏的「智能收藏夹」区域可以创建和管理。

2.12.1 规则结构

每条智能收藏夹的规则以 JSON 树存储，支持嵌套的 AND/OR 逻辑。规则树的结构：

```
{  
  "op": "and",  
  "conditions": [  
    { "field": "read_status", "op": "eq", "value": "unread" },  
    { "field": "year", "op": "gte", "value": 2024 },  
    { "field": "tags", "op": "includes", "value": "transformer" },  
    { "op": "or", "conditions": [  
      { "field": "folder", "op": "in", "value": "ML" },  
      { "field": "title", "op": "contains", "value": "attention" }  
    ]  
  ]  
}
```

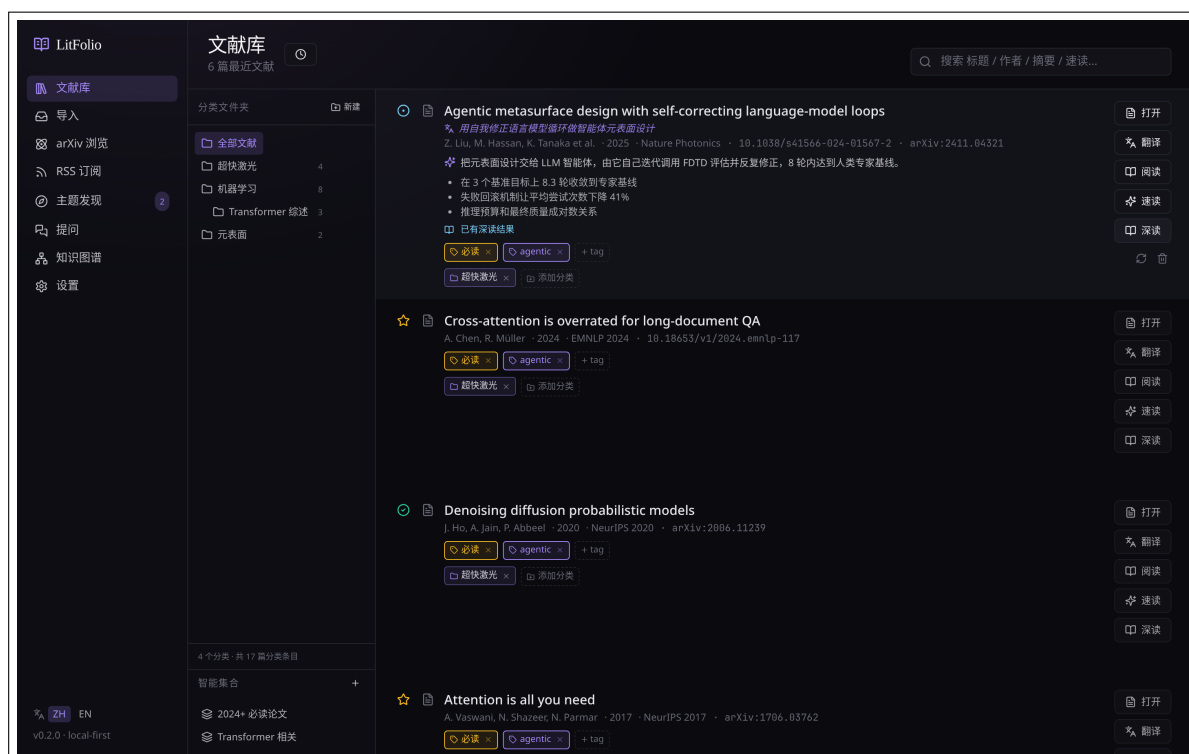


图 2.4 智能收藏夹创建对话框。通过组合条件筛选论文，结果实时更新。

```

    ]}
  ]
}

```

2.12.2 SQL 翻译引擎

查询时，规则树被递归翻译为 SQL WHERE 子句：

1. 叶节点——每个条件翻译为一个 SQL 片段：

- `read_status` → `p.read_status = '<value>'`
- `year + gte/lte` → `p.year ≥ <value>` 或 `p.year ≤ <value>`
- `tags + includes` → `EXISTS (SELECT 1 FROM paper_tags pt JOIN tags t ON pt.tag_id = t.id WHERE pt.paper_id = p.id AND t.name = '<value>')`
- `folder + in` → `EXISTS (SELECT 1 FROM paper_folders pf JOIN folders f ON pf.folder_id = f.id WHERE pf.paper_id = p.id AND f.name LIKE '<value>%') (支持前缀匹配以包含子文件夹)`
- `title + contains` → `p.title LIKE '%<value>%'`
- `authors + contains` → `p.authors LIKE '%<value>%'`

2. 组合节点——`op`: "and" 翻译为 `(cond1 AND cond2 AND ...)`, `op`: "or" 翻译为 `(cond1 OR cond2 OR ...)`。嵌套深度不限。

3. **最终查询**——拼接为 `SELECT p.* FROM papers p WHERE < 规则 > ORDER BY p.added_at DESC`。

2.12.3 实时更新

智能收藏夹不是快照——它是每次选中时实时查询的。所以当一篇论文的标签或状态变更后，下次点击该收藏夹会立即反映变化。这与普通文件夹的语义完全不同：普通文件夹需要你手动拖入/拖出，智能收藏夹只关心规则是否匹配。

2.12.4 性能

规则翻译后的 SQL 走 SQLite 索引。对于标签和文件夹条件，`paper_tags` 和 `paper_folders` 表有 `paper_id` 索引，`EXISTS` 子查询很快。10 个智能收藏夹不会对侧栏加载造成可感知的延迟——因为只有被选中的那个才会执行查询，其他的只显示名称和规则描述。

提示 智能收藏夹适合长期维护的视角，比如「近两年未读的机器学习论文」或「所有必读论文」。临时筛选用搜索框更方便。规则一旦创建可以随时编辑——修改规则后匹配结果立即更新。

2.13 阅读队列

阅读队列是一个独立于文件夹/标签的优先级排序工具，解决「有 20 篇想读但需要排个先后」的问题。在文献库侧栏点击「阅读队列」进入。

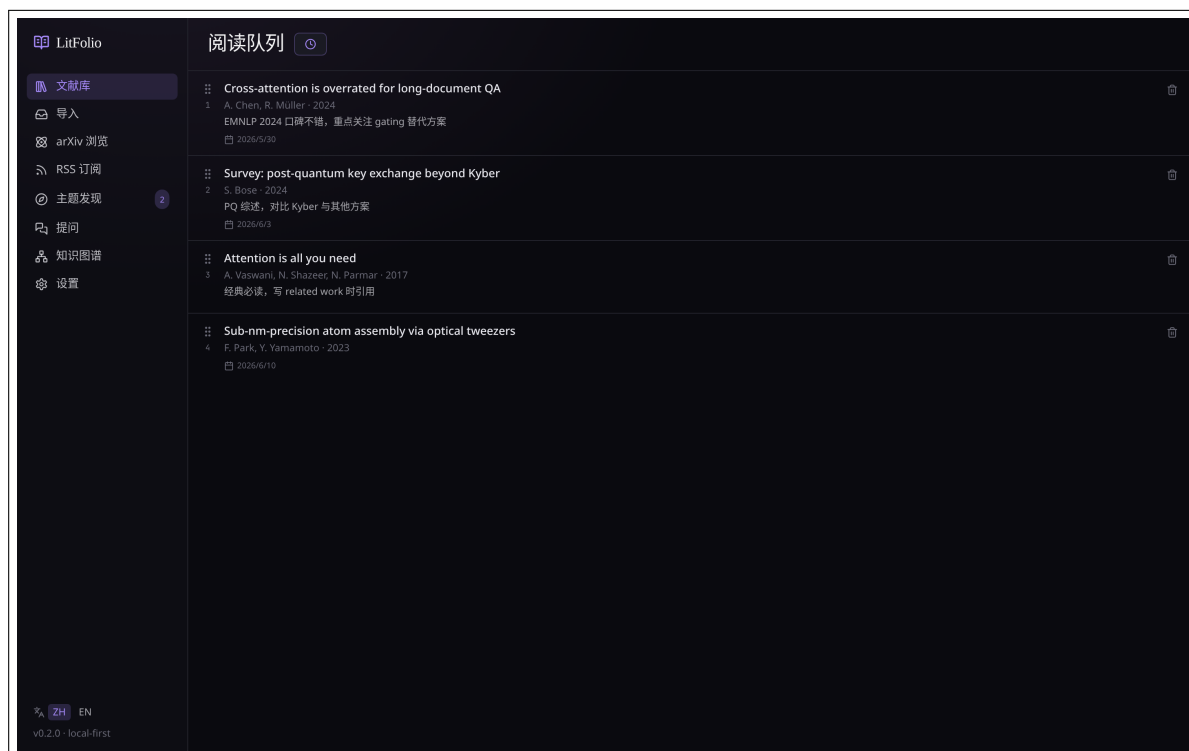


图 2.5 阅读队列。拖拽排序，设置目标日期，超期论文高亮提示。

2.13.1 队列数据模型

队列存储在 `reading_queue` 表中，每行包含：

- `paper_id`——关联论文（主键，一篇论文只能在队列中出现一次）。
- `priority`——整数优先级，由拖拽排序位置决定（从 0 开始递增）。
- `target_date`——可选的 Unix 时间戳，表示计划阅读日期。
- `note`——可选的备注文本。
- `added_at`——入队时间。

2.13.2 拖拽排序机制

前端使用 HTML5 Drag and Drop API 实现拖拽排序。拖拽完成时：

1. 计算拖拽目标的新位置索引。
2. 重新分配所有队列项的 `priority` 值（从 0 开始递增，步长 1）。
3. 调用 `queue_reorder` 命令批量更新数据库。

排序结果跨会话持久化——关闭应用再打开，顺序不变。

2.13.3 目标日期与超期提醒

目标日期是可选的。设置了目标日期的论文，如果当前日期超过目标日期，该行会以琥珀色高亮显示（与 `warnbox` 使用相同的 `amber` 色值）。超期只是视觉提示，不会自动改变论文的任何状态。

2.13.4 自动状态更新

当你在阅读器中打开一篇队列中的论文时，系统会自动将该论文的 `read_status` 从 `unread` 切换为 `reading`。这个转换是幂等的——如果已经是 `reading` 或更高状态，不做任何操作。

2.13.5 队列与文件夹/标签的关系

队列是完全独立的维度：

- 一篇论文可以同时在任何文件夹、挂任何标签、处于任何阅读状态，仍然可以加入队列。
- 从队列中移除论文不会删除论文本身，也不会改变其文件夹/标签/状态。
- 队列中引用的论文如果被删除，队列条目会被级联删除。

2.14 论文去重

LitFolio 在导入时自动检测重复论文，避免同一论文因不同来源（arXiv ID / DOI / PDF 文件导入）而出现多份副本。

2.14.1 三级匹配策略

检测按优先级依次执行，命中即停：

1. **DOI 精确匹配**——查询 `SELECT id FROM papers WHERE doi = ?`。DOI 是全球唯一的数字对象标识符，匹配率 100%。
2. **arXiv ID 精确匹配**——查询 `SELECT id FROM papers WHERE arxiv_id = ?`。同一论文不同版本（v1/v2）在去比较时去掉版本后缀再匹配。
3. **标题相似度匹配**——使用归一化 Levenshtein 编辑距离。设两标题为 s_1, s_2 ，编辑距离为 $d(s_1, s_2)$ （插入/删除/替换一个字符各计 1），归一化公式为：

$$D(s_1, s_2) = \frac{d(s_1, s_2)}{\max(|s_1|, |s_2|)}$$

当 $D < 0.15$ 时判定为重复。这意味着两标题最多差 15% 的字符——足以容忍大小写差异（Attention Is All You Need vs attention is all you need）、标点差异（连字符 vs 空格）、以及少量 OCR 错误。

2.14.2 标题预处理

在计算编辑距离之前，两个标题都经过预处理：

1. 转小写。
2. 移除所有非字母数字字符（保留空格）。
3. 将连续空格合并为单个空格。
4. 去除首尾空格。

这保证了 "Attention-Is-All-You-Need" 和 "attention is all you need" 的距离为 0（完全匹配）。

2.14.3 全库扫描

设置页的「查找重复」按钮执行一次全库扫描。算法为 $O(n^2)$ 标题比较（ n 为论文数），但有两个优化：

- **DOI/arXiv ID 先筛**——有 DOI 的论文先按 DOI 分桶，同桶内才做标题比较；有 arXiv ID 的同理。这大幅减少需要做字符串比较的论文对数量。

- **长度预筛**——如果两标题长度差超过 15%, 直接跳过编辑距离计算。这是一个保守的剪枝: $|s_1| - |s_2| > 0.15 \times \max(|s_1|, |s_2|)$ 时 D 必然 > 0.15 。

1000 篇论文的全库扫描在普通硬件上约需 2-3 秒。

2.14.4 合并流程

检测到重复时弹出合并对话框。你选择保留哪条记录（通常选元数据更完整的那条），系统执行以下合并：

1. 将被合并论文的高亮、笔记、术语、标签、文件夹归属全部转移到保留论文。
2. 将被合并论文的阅读状态取两者中更高的（must > read > reading > unread）。
3. 将被合并论文的 TL;DR、深读、翻译字段填入保留论文的空缺处（不覆盖已有值）。
4. 删除被合并论文的数据库行和 `papers/<id>/` 目录。

注意 合并操作不可撤销。被合并论文的 PDF 文件会被移到 `backups/deleted/`，但元数据直接删除。如果你不确定应该保留哪条，可以先取消合并，手动检查两条记录的完整性后再操作。

导入文献

「导入」页（/import）汇集了三种入口：arXiv/DOI 元数据、本地 PDF、Semantic Scholar 搜索。三者最终都落到 `papers/<id>/` 目录与 SQLite 行。

3.1 arXiv ID / DOI

最常见的入库路径。流程是**两步**：

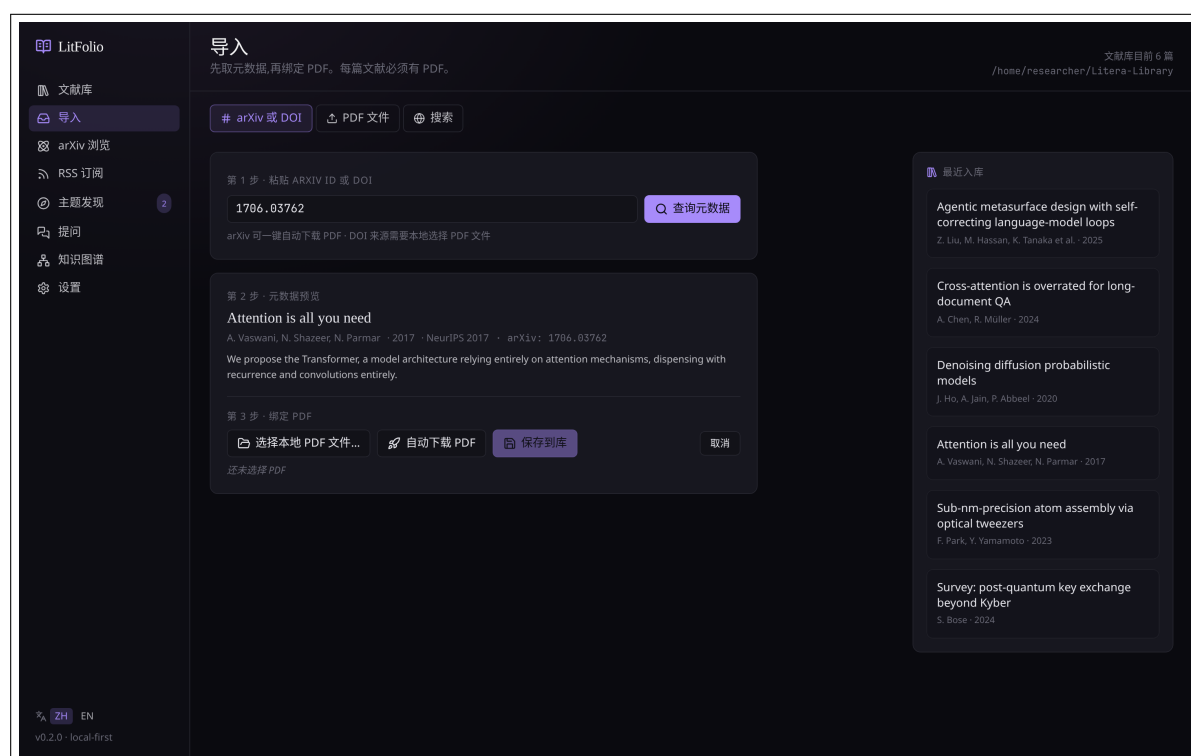


图 3.1 arXiv/DOI tab。粘贴 ID 或 DOI 后点「查询元数据」拿到标题/作者/摘要，再点「入库」落盘。

1. **查询元数据**——粘贴形如 1706.03762 的 arXiv ID 或 10.1038/s41586-024-... 的 DOI，点查询。LitFolio 会去 arXiv API 或 Crossref 抓元数据，立刻展示标题、作者、摘要。
2. **入库**——确认无误后点「入库」。LitFolio 会同时尝试下载 PDF（arXiv 直接拿，DOI 走开放访问的 unpaywall 链路；失败也不阻塞入库）。

提示 arXiv ID 既支持新格式(2402.12345)也支持老格式(cs/0501001)。带版本号的 2402.12345v2 也会被正确解析。

3.2 本地 PDF 文件

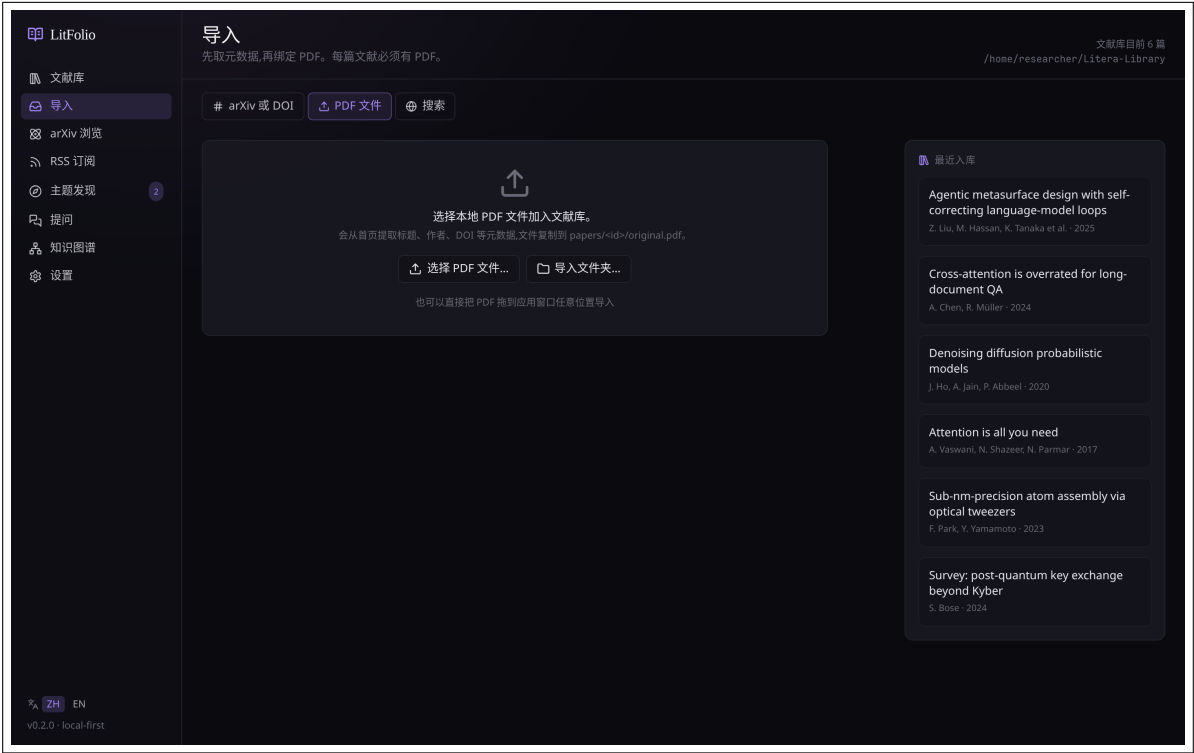


图 3.2 PDF 文件 tab。可以挑单个 PDF，也可以批量选目录；选好后能编辑标题/作者再入库。

- **单个 PDF**——选文件后 LitFolio 会跑一遍轻量元数据抽取（PDF 第一页 + DOI 正则），猜出标题/作者；你可以在表单里手改。
- **拖拽**——把 PDF 直接拖到 LitFolio 任意页面，会跳到导入流程。

3.2.1 PDF 元数据提取策略

本地 PDF 入库时，LitFolio 按以下顺序尝试提取元数据：

1. **PDF Info Dictionary**——用 `lopdf` 读取 `/Title`、`/Author`、`/Subject` 字段。文本解码同时尝试 UTF-8 和 UTF-16 BE（带 BOM），因为学术 PDF 两种编码都常见。
2. **第一页 DOI 正则**——如果 Info Dictionary 里没有 DOI，用正则 `10.\d{4,9}/\s+` 扫描第一页文本，匹配后去掉末尾的标点（.,;] 等）。
3. **标题猜测**——如果仍然没有标题，取第一页第一行长度 ≥ 12 且含字母的行作为标题。
4. **文件名兜底**——以上全失败时，用 PDF 文件名（去掉扩展名）作为标题。

作者字段按逗号、分号和「and」三种分隔符拆分（适配 BibTeX 和自然语言两种写法）。入库时还会计算文件 SHA-256 哈希用于去重。

3.3 Semantic Scholar 搜索

如果你只记得标题片段、关键词、或想搜某作者的全部论文：

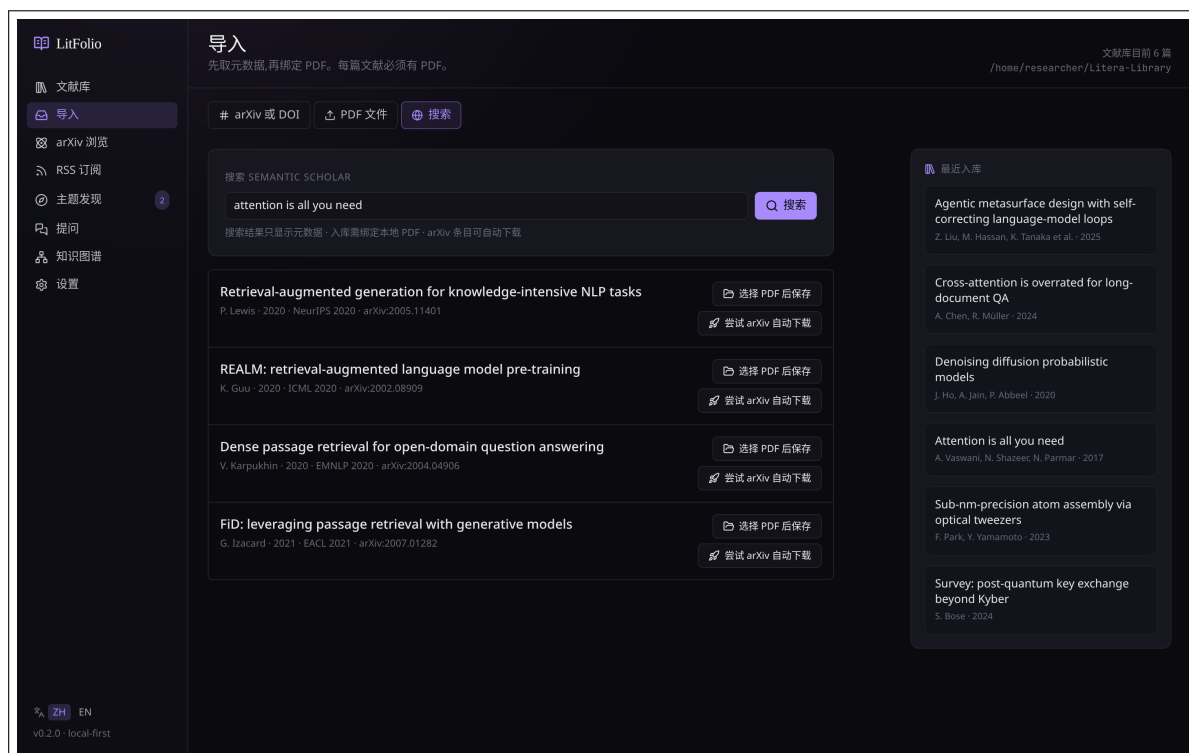


图 3.3 搜索 tab。结果按引用数排序，每条都能直接「入库」（拉元数据 + 尝试下载 PDF）。

LitFolio 调 Semantic Scholar Graph API，每页返回 25 条；命中后点「入库」会立刻拉该条目的完整元数据并尝试下载 PDF。

注意 Semantic Scholar 公共 API 有 QPS 限制，频繁搜索可能 429。如果你的研究方向需要大量检索，建议在它们网站申请一个免费的 API key，填到「设置 → API Keys」里。

3.4 批量文件夹导入

除了单个 PDF 导入，LitFolio 支持导入整个文件夹中的 PDF 文件。在「导入」页点击「导入文件夹」按钮，选择一个目录。

3.4.1 递归扫描

选中文件夹后，LitFolio 递归扫描所有子目录中的 .pdf 文件。扫描结果以列表形式展示，每个文件显示：

- 文件名（作为标题猜测的兜底来源）。
- 文件大小。
- 提取状态（等待中 / 处理中 / 成功 / 失败）。

3.4.2 文件名启发式解析

对于批量导入的 PDF，LitFolio 会尝试从文件名中提取元数据。常见命名模式：

Author_Year_Title.pdf 按下划线拆分：第一段为作者，第二段为年份（4 位数字），其余为标题。

Author - Title (Year).pdf 按 - 拆分作者和标题，从括号中提取年份。

2024 - Author - Title.pdf 年份在前的变体。

启发式解析的准确率约 70%——对于无法解析的文件，标题使用文件名（去掉扩展名），作者和年份留空，你可以在入库后手动补充。

3.4.3 并发处理与进度追踪

批量导入使用 4 路并发处理（可配置）。进度通过事件流实时推送到前端：

- 进度条显示「处理中 23/150…（3 失败）」。
- 失败文件单独列出，附带失败原因（PDF 损坏、无文本层、元数据提取失败等）。
- 处理完成后显示汇总：成功 N 篇、失败 N 篇、跳过 N 篇（已存在的重复论文）。

提示 批量导入 100 篇 PDF 通常在 5 分钟内完成。如果某些 PDF 是扫描版（无文本层），LitFolio 仍然会入库（用文件名作为标题），但全文搜索和 RAG 召回将无法覆盖这些论文的内容。可以用 `ocrmypdf` 预处理扫描版 PDF 后重新绑定。

3.5 从 RSS / arXiv 浏览跳转

第 5、6 章里讲的 arXiv 浏览和 RSS 订阅，详情抽屉里都有「入库」按钮，效果等同于在「导入」页粘贴 arXiv ID 后入库。也就是说：你不必专门去「导入」页，刷 RSS 时看到喜欢的直接入。

阅读器

阅读器是 LitFolio 第二常驻的页面。从主列表点  或「在阅读器中打开」即可进入。

4.1 三栏布局

4.1.1 PDF 全文提取与缓存

阅读器加载 PDF 时，PDF.js 会在前端完成全文文本提取（它比后端 `lopdf` 对现代学术 PDF 的 CMap 编码、嵌入字体子集、ToUnicode 表处理更可靠）。提取出的文本通过 IPC 发送给后端，缓存到 `papers/<id>/text.txt`。后续的术语提取和 FTS5 索引都直接读这个缓存文件——只要 `text.txt` 的修改时间比 `paper.pdf` 新就复用，否则用 `lopdf` 重新提取。这意味着：术语生成不需要重新解析 PDF，瞬间可用。

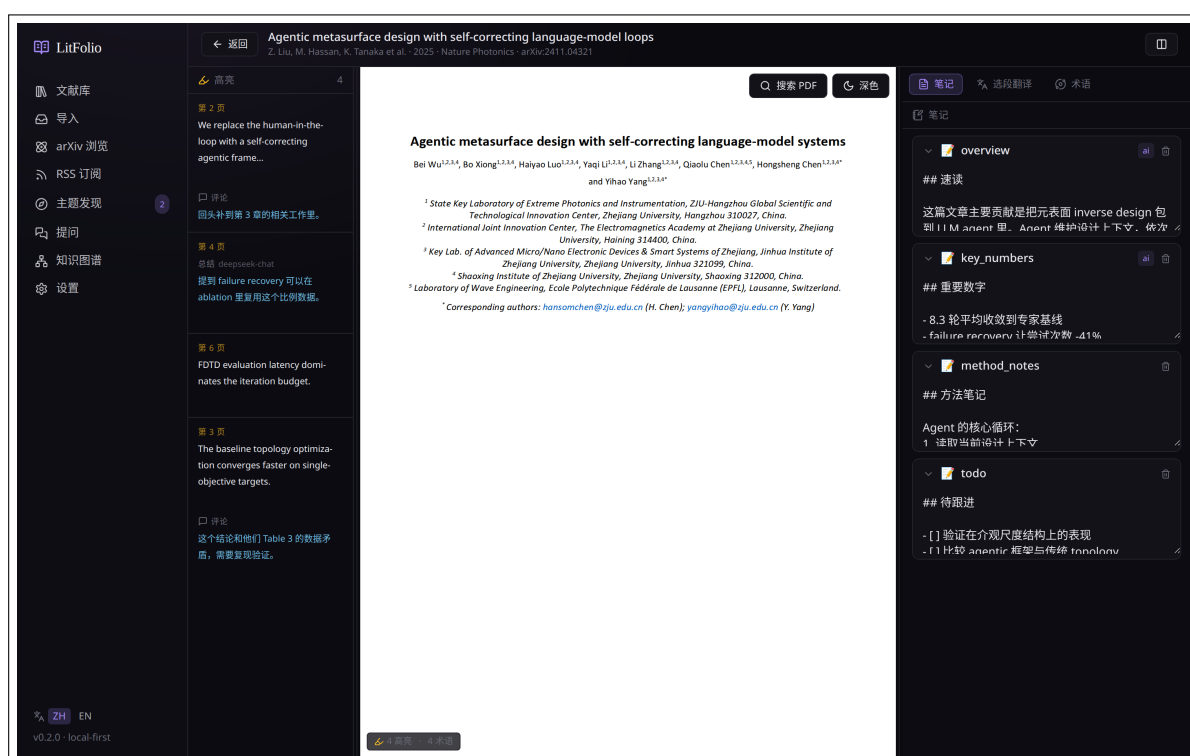


图 4.1 阅读器三栏。左：高亮列表；中：PDF.js 渲染的文档；右：笔记/选段翻译/术语三 tab 切换。三栏宽度可拖拽调节，状态自动持久化。

- **左栏**——本论文已有的全部高亮（按页码排序），点任一条跳到对应位置。
- **中栏**——PDF 主体。基于 PDF.js + react-pdf-highlighter；继承系统深色模式，渲染时会反相显示。
- **右栏**——三 tab 在线工具：**笔记**（Markdown）、**选段翻译**、**术语**。

4.2 高亮：文本选区 + 区域

两种高亮方式：

选区高亮 像普通 PDF 一样选中一段文字，松开鼠标即弹出小气泡，点「高亮」即可。选区文本会被 OCR 不到的字符（连字、转行）规整成清晰文字写入高亮表。

区域高亮 (**Alt+** 拖拽) 按住 Alt 在 PDF 上拖一个矩形，会把矩形截图保存为高亮——适合捕捉公式块、图表、流程图。

左栏每条高亮可以加评论（点铅笔图标）；评论用 Markdown 写。

4.3 选段翻译

PDF 中选中正文后，切到右栏「选段翻译」tab，点「翻译选段」：

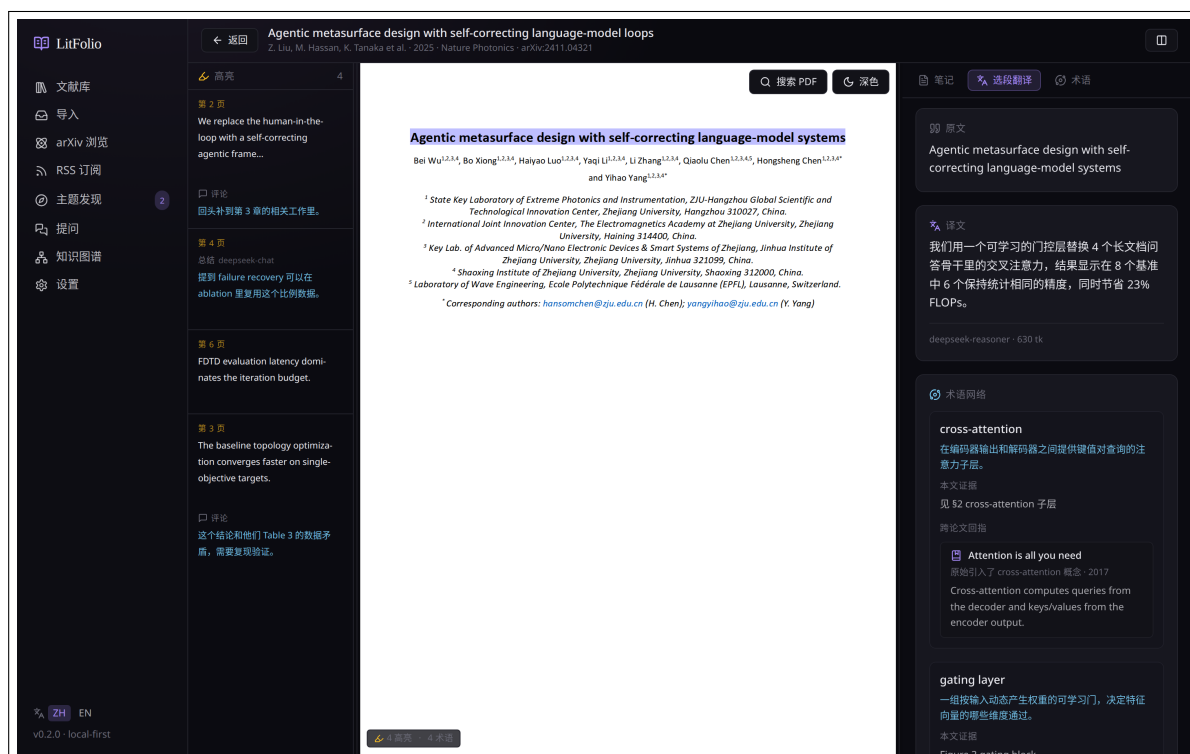


图 4.2 选段翻译。除了双语对照，还把段内出现的术语用紫色 chip 列出来——点击任一术语跳到「术语」tab 看完整定义。

输出结构是「双语对照 + 术语抽取 + 跨论文回指」三段。

选段翻译的内部流程

选段翻译并不是简单地调一次翻译 API，而是分三步执行：

1. **术语匹配**——先检查选段文本中是否包含当前论文已有的术语（按归一化后的术语做大小写不敏感的子串匹配）。命中则直接复用已有的定义和证据片段；未命中则走实时提取：用三组正则分别捕获全大写缩写（如 SSIM）、Title Case 专有名词（如 MetaDesigner）和 1-3 词名词短语，再经 `is_term_candidate` 过滤器筛选。
2. **跨论文关联**——对每个术语在全库做 FTS5 搜索（最多返回 12 条），排除当前论文后保留最多 3 篇关联论文。关联关系根据术语出现在目标论文的哪个段落来分类：出现在 `method` 段 → 「方法实现」、`comparison` 段 → 「对比讨论」、其余 → 「相关提及」。
3. **LLM 翻译**——将选段文本（截断至 2000 字符以内）交给翻译模型。`prompt` 中会注入一份术语 `glossary`（包含匹配到的术语及其本地定义），要求 LLM 保留术语英文原文，并在首次出现时以 `Term [中文释义]` 的格式标注。这样译文既通顺又不丢失术语锚点。

最终前端将三项结果拼合展示：双语对照 + 紫色术语 `chip` 列表 + 跨论文链接。

- **双语对照**——逐段渲染原文与译文。
- **术语**——从这段里抽出的领域术语，每个带一句话本文解释；如果其它论文也用到过同一术语，会标「跨论文复用 N」。
- **跨论文回指**——当前论文之前讨论同一概念的位置（页码 + 行）；常用于长文献的「我前面是不是已经看过这个？」场景。

4.4 术语表与术语网络

「生成术语表」走 LLM `term` 任务（默认绑你设置的术语模型）。生成后术语会缓存到 `papers/<id>/` 目录与 SQLite 表 `terms`，供后续选段翻译复用。

4.4.1 术语提取算法

术语生成并不是简单地把全文丢给 LLM 让它列术语。LitFolio 采用**本地统计筛选 + LLM 定义**的两阶段架构：先用不调网络的 TF-IDF 统计方法从全文中筛出候选术语，再让 LLM 只为这些候选写一句话定义。这样做的好处是：省 `token`（不需要让 LLM 扫全文），且候选列表可复用于选段翻译时的术语匹配。

候选提取：加权 $TF \times IDF \times$ 多因子评分

整个候选流程分四步：

1. **加权分段**——论文的各个部分被赋予不同权重（标题 3.0、摘要 2.0、方法 2.0、研究问题 1.5、数据集 1.5、对比 1.0、局限 1.0、TL;DR 1.0、PDF 全文 0.4）。术语在标题或摘要中出现时权重更高，在正文中出现时权重较低。

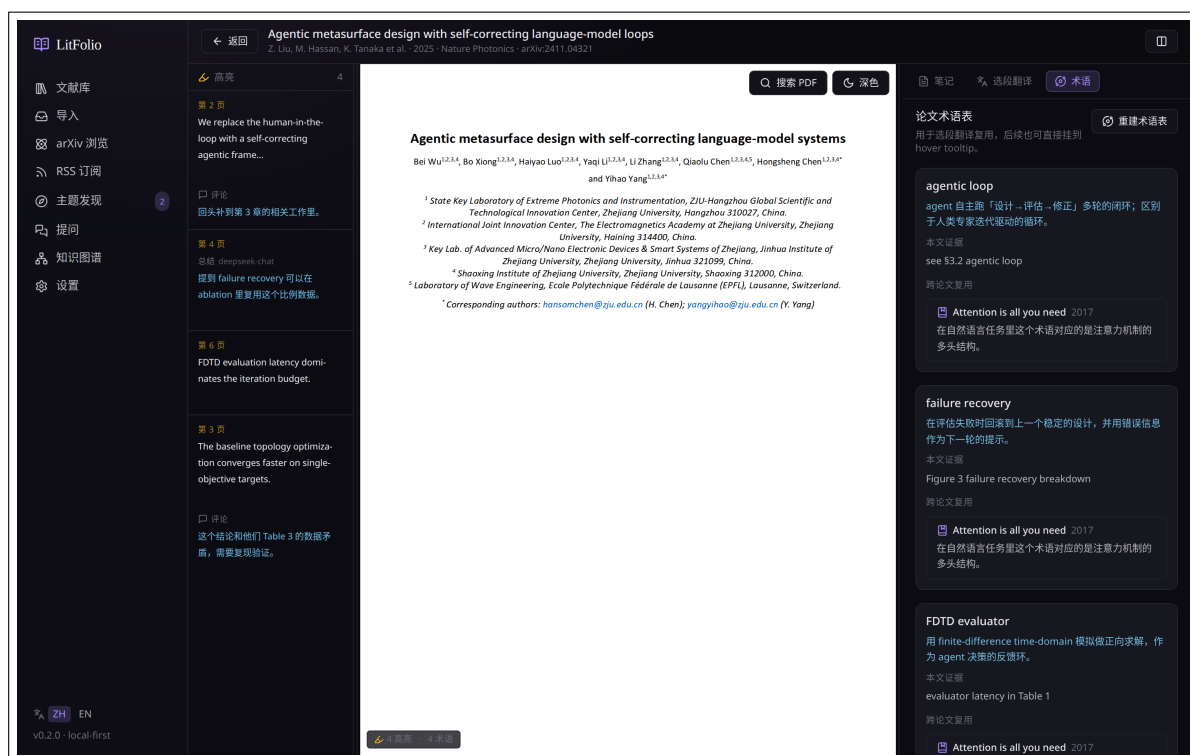


图 4.3 术语 tab。生成本文术语表后，每个术语展示「本文证据 / 跨论文复用」两节，便于跨论文复用与一致性检查。

- 正则提取 + 过滤**——对每段文本做正则匹配：默认只捕获单个 token（专有名词、全大写缩写、连字符复合词）；对「Full Name (ACRO)」缩写对则走专门的缩写识别路径，允许全称最多 5 个词。每个候选必须通过 `is_term_candidate` 过滤器：首尾词不能是虚词（with/from/of...），3+ 词短语中间不能有连词/介词（and/or/with...），纯小写单词至少 4 字符且必须有大写或连字符才保留。
- TF-IDF 计算**——term frequency 是候选在各加权段中出现次数乘以段权重的累加；document frequency 是候选在库里最近 500 篇论文中出现的篇数。IDF 公式为 $\ln((N + 1)/(df + 1)) + 1$ 。再乘以三个修正因子：
 - section_hits** ($\sqrt{\min(3, \text{出现段数})}$)——出现在越多不同段落里越可信。
 - surface_quality**——全大写缩写 $\times 2.0$ 、Title Case 多词 $\times 1.6$ 、含连字符 $\times 1.3$ 、纯小写多词 $\times 0.7$ 。表面形式越像专业术语得分越高。
 - abbrev_bonus** ($\times 1.8$)——如果候选来自缩写对路径，额外加权。
- 排序截断**——总分 ≥ 0.9 的候选按降序排列，取前 12 个。低于阈值的直接丢弃（比如一次出现的纯小写 bigram 大约 0.7 分，刚好被过滤掉）。

缩写对识别

对于形如「Structural Similarity Index Measure (SSIM)」的文本，LitFolio 用专门的正则捕获全称 + 缩写，然后做两项检查：

- **首字母匹配**——取全称各词首字母，与缩写逐字符做贪心匹配，至少一半命中才接受。连字符被当作词界（「Retrieval-Augmented Generation」→ RAG 可以匹配）。
- **全称精炼**——如果正则捕获的全称比缩写长，会从尾部截取恰好等于缩写长度的词序列做首字母精确匹配；截不中则先去掉停用词（and/the/of）再试。例如「average structural similarity index measure (SSIM)」会精炼为「structural similarity index measure」。

缩写对只有缩写进入最终术语列表（全称不单独出现），但全称会保存为 LLM 定义的上下文。

作者名过滤

一个容易被忽略的噪声源是作者姓氏——「Chen」「Yang」「Li」这些常见姓氏在论文的作者栏里高频出现，满足 Title Case 专有名词的条件，会排到术语候选前面。LitFolio 在提取前先把当前论文所有作者的姓氏收集到一个 HashSet 中（按空格、连字符、句点拆分后小写化），候选归一化后如果命中就直接跳过。这对中文作者的拼音姓氏尤其重要——「Chen」「Wang」在作者栏里出现频率极高，不加过滤会把它们当成术语排到前面。

LLM 定义生成

候选确定后，把每个术语的「表面形式 + 全称（如有）+ 证据片段」拼成 prompt 交给 LLM，要求返回 JSON 格式的中文一句话定义。如果 LLM 调用失败或返回空，使用 fallback：本文将 <term> 放在当前论证或方法上下文中使用。

提示 术语生成耗 token 比 TL;DR 多得多（要扫全文），建议绑性价比高的国内低价档对话模型，或者本地跑一个 7B 以上的开源模型。

4.5 结构化笔记卡片

「笔记」tab 已从单一文本框升级为**结构化卡片系统**。每篇论文默认有五个独立编辑的笔记分区，存储在 paper_note_sections 表中。

4.5.1 默认分区

Problem 这篇论文解决什么问题。

Method 提出了什么方法。

Key Numbers 关键实验数据或指标。

Limitations 局限与未解决的问题。

Thoughts 你自己的想法和评论。

每个分区独立编辑、可折叠、可拖拽排序。排序结果通过 `sort_order` 字段持久化。你可以添加自定义分区（如「实验想法」「相关工作」），自定义分区与默认分区享有相同的编辑和排序能力。

4.5.2 AI 自动填充机制

- 分区有一个 `source` 字段，标记内容来源：
 - user** 你手动写入的内容。
 - ai:tldr** 由速读（TL;DR）自动填充。
 - ai:quick_read** 由深读（Quick Read）自动填充。

填充规则是只填空、不覆盖：

- 运行速读时，LLM 返回的 `key_findings` 数组被写入 `Key Numbers` 分区——但仅当该分区的 `content` 为空且 `source` 不是 `user` 时。
- 运行深读时，LLM 返回的四段 JSON 被映射到对应分区：`problem` → `Problem`、`method` → `Method`、`limitations` → `Limitations`。同样只填空分区。
- 如果你手动编辑过分区（`source = user`），AI 永远不会覆盖它。即使你重新运行速读或深读，该分区也会被跳过。
- AI 填充的内容追加时间戳前缀：`[2025-05-27 AI:TLDR] < 内容 >`。

4.5.3 分区与全文搜索

所有分区的 `content` 字段被纳入 `papers_fts` FTS5 索引。这意味着你可以在搜索框中输入某个分区里写过的关键词来找到对应论文——即使标题和摘要中完全没有出现这个词。

提示 分区的拖拽排序通过 HTML5 Drag and Drop 实现，排序结果写入 `sort_order` 字段。默认排序为 `Problem` → `Method` → `Key Numbers` → `Limitations` → `Thoughts`。

4.6 标注颜色语义

高亮颜色可以赋予语义含义，存储在 `highlights.label` 字段中。在「设置 → 标注颜色」中配置颜色-标签映射。默认映射：

颜色	标签	语义
黄色	Key Finding	关键发现、重要结论
紫色	Method	方法要点、算法细节
蓝色	To Verify	需要验证的声明、存疑数据

颜色	标签	语义
红色	Disagree	不认同的观点、有争议的假设
绿色	Citable	可直接引用的精彩表述

4.6.1 工作流

1. **创建高亮**——选中文本后弹出气泡菜单，菜单中每种颜色旁显示对应标签名（如「黄色·Key Finding」）。选择颜色即同时设置标签。
2. **筛选高亮**——阅读器左栏顶部增加一行标签筛选 chip。点击某个 chip 只显示该标签的高亮；再次点击取消筛选。可以多选（OR 语义）。
3. **AI 集成**——高亮摘要功能（对选中高亮调 LLM 生成总结）支持按标签过滤。例如只对「Key Finding」标签的高亮做摘要，忽略「To Verify」的噪声。
4. **导出集成**——Markdown 导出时，高亮按标签分组输出，每个标签作为二级标题。

4.6.2 自定义标签

你可以在设置中修改颜色-标签映射：更改标签名、添加新颜色、删除不使用的映射。已有的高亮不受影响——标签名存储在每条高亮记录中，而非引用全局配置。

4.7 Markdown 笔记

「笔记」tab 底部仍保留自由文本 Markdown 编辑区，自动保存到 `papers/<id>/note.md`。LitFolio 不强制你按某种模板写——你可以用任何 Markdown 语法、嵌任意图片链接，文件后续都跟 PDF 一起被 WebDAV 同步。

4.8 PDF 搜索

- Ctrl+F（macOS: Cmd+F）打开搜索框：
- Enter 下一个，Shift+Enter 上一个
 - 匹配数会显示在搜索框右侧
 - Esc 关闭

4.9 页面快捷键

阅读器中的其它常用键：

- **j** / **k** —— 下一页 / 上一页
- **1** / **2** / **3** —— 切换右栏 tab
- **[** / **]** —— 拖动左/右栏宽度（步进 5%）

完整快捷键表见 **第 11 章**。

第二部分

发现与综合

arXiv 浏览

/browse 路由是一个为研究者「逛 arXiv」而设计的页面。它按 arXiv 官方分类树组织，让你在不预先有关键词的情况下浏览某领域的最新论文。

5.1 分类树

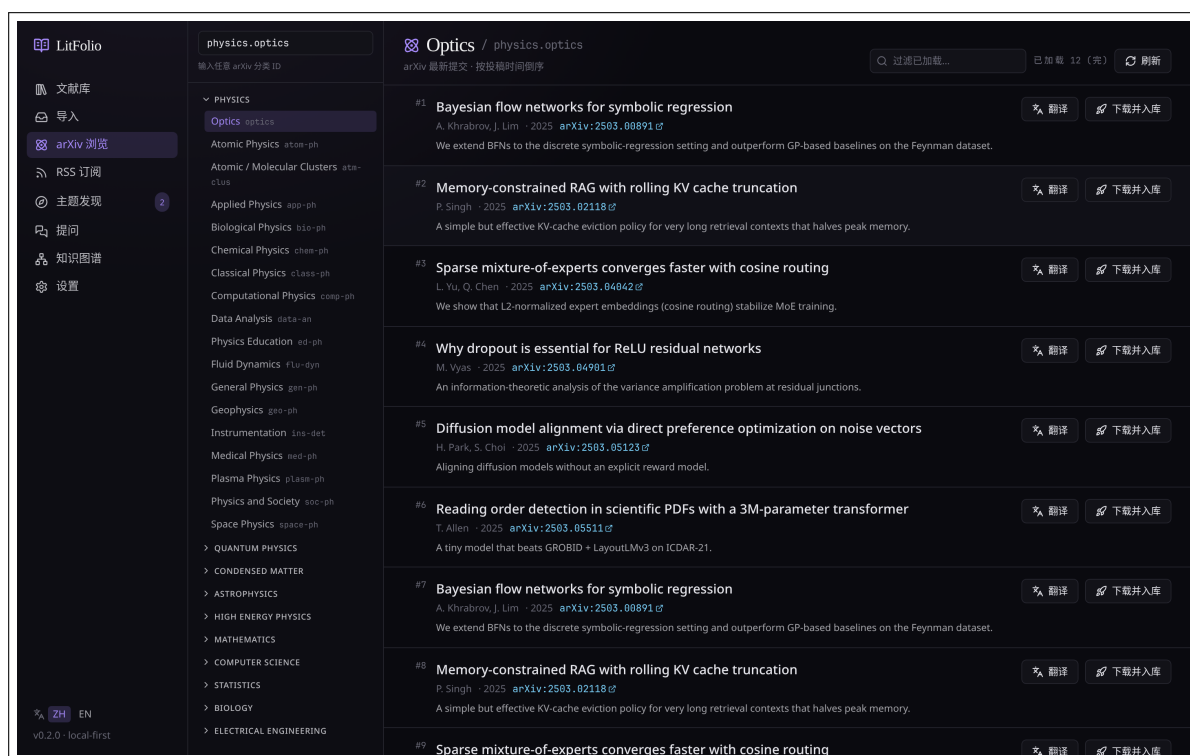


图 5.1 arXiv 浏览。左侧分类树（按 cs / math / physics 等顶层归类），主区按选中分类显示最新论文，支持往前翻页和入库。

LitFolio 内置了 arXiv 全部 150+ 分类的中英文映射；点任一叶子节点（如 cs.LG 机器学习、physics.optics 光学）会发起一次 arXiv API 请求拉取最新 50 条。

5.2 翻页与「已穷尽」

往下滚到底点「加载更多」会继续抓更多页（每页 50 条）。当某分类发布量较小、连续两次返回都没有新条目时，LitFolio 会显示「已穷尽」并停止请求——这是为避免无意义的 API 调用，不是 bug。

5.3 入库

点任一论文打开详情抽屉，可看到完整摘要、作者、提交日期、原始 arXiv 链接。抽屉右下角的「入库」按钮等同于第 3 章里 arXiv ID 入库流程，区别只是这里 ID 已经填好了。

提示 arXiv 浏览页本身**不会**把列表保存到本地数据库——它每次都是实时 API。如果你想长期跟某分类的新论文，建议改用 RSS 订阅（见下章），可离线、可翻译、可标记已读。

RSS 订阅

/feeds 路由让你以 RSS 方式跟踪 arXiv 子分类、期刊新文章、研究组主页。所有拉到的条目会落 SQLite，未读计数与已读状态都本地保存。

6.1 订阅源管理

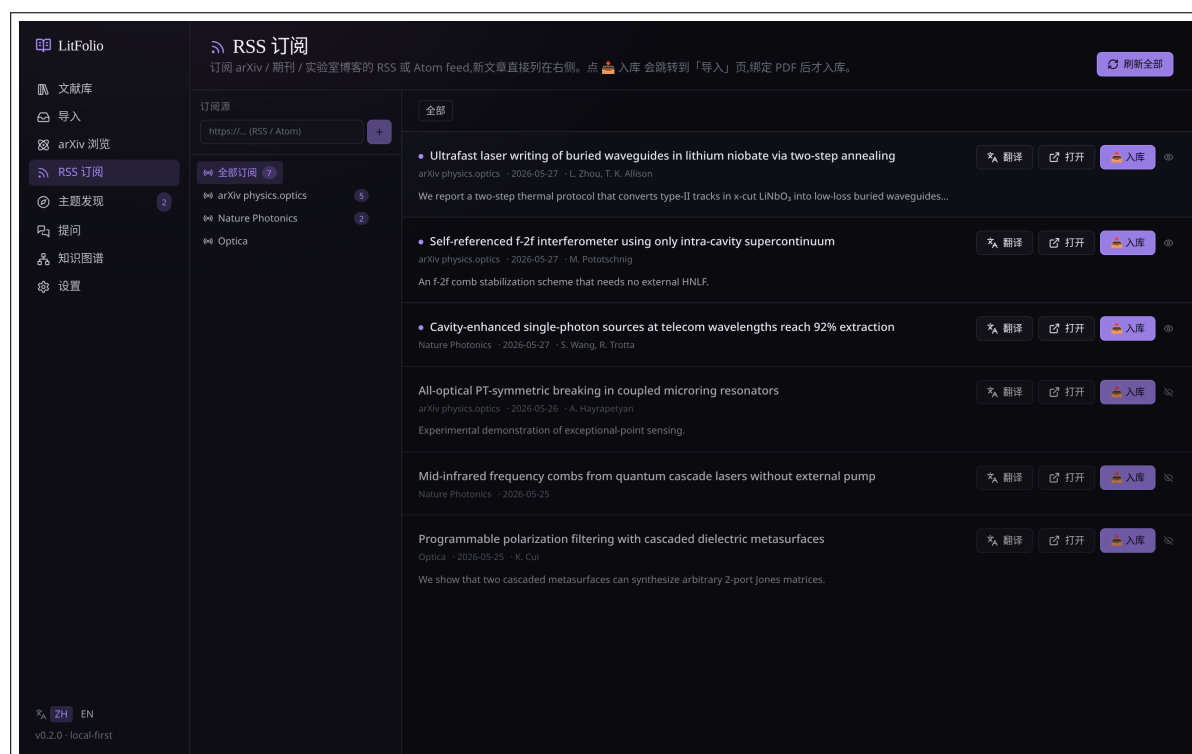


图 6.1 订阅源列表。左：源（arXiv cs.LG / Nature Photonics / Optica 等）；右：选中源的最新条目，带未读徽章、翻译按钮、入库按钮。

新增订阅源：点「+」按钮，填 RSS URL 与显示名即可。LitFolio 不限定 feed 来源，只要是合法 RSS/Atom 都行。常用源参见附录 C。

6.2 条件 GET 刷新

LitFolio 用条件 GET (If-Modified-Since / ETag) 拉 feed，远端没更新时会返回 304 Not Modified——这是为什么有时点「刷新」秒回「已是最新」。这一行为对你来说意味着：

- 频繁点刷新是**廉价的**，不会触发对方限流。
- 真正的「没新东西」与「网络/对端故障」都会显示同一文案，需要看错误徽章区分。

6.3 条目详情与翻译

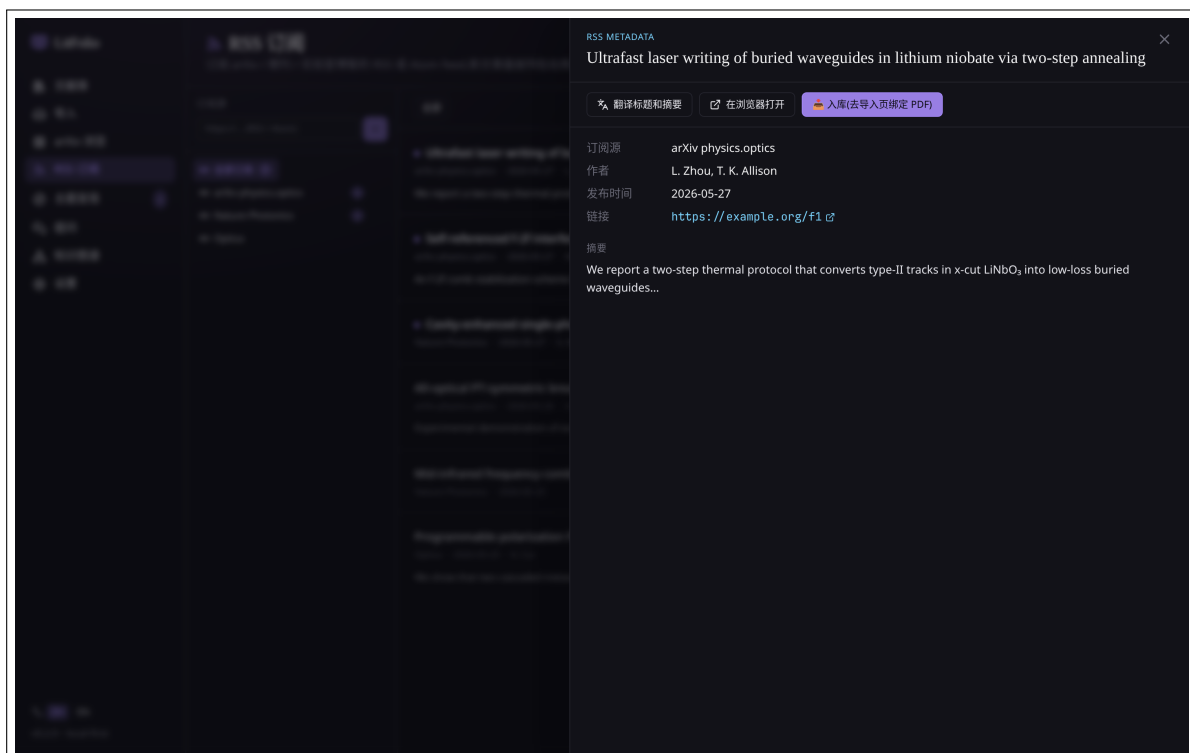


图 6.2 选中某条 feed item 后右侧抽屉显示完整摘要；可一键翻译，可标记已读，可直接入库（等同于到「导入」页粘 arXiv ID）。

「翻译」按钮和文献库一致：调 `translate` 任务（绑你设置的翻译模型）。结果同样缓存到 SQLite `feed_items` 表，下次打开秒显示。

注意 RSS 翻译每个条目都会消耗 token；如果你订阅的源更新很快（如多个 arXiv 分类），建议把翻译任务绑给本地 Ollama 上的开源对话模型或者国内低价 API，避免开销失控。

6.4 从 feed item 直接入库

详情抽屉的「入库」按钮和第 3 章 arXiv ID 入库流程完全一致——拉完整元数据 + 尝试下载 PDF。入库后该条目仍留在 feed 列表里，只是会显示「已入库」标记。

主题发现

/topic 路由是 LitFolio 的进阶能力之一，分两个 tab：**搜索召回**与 **综述生成**。两者解决的问题不同。

7.1 两 tab 心智模型

搜索召回 你已经有大致研究方向，想从已经入库的论文里把相关的全捞出来。流程：扩写查询 → 多路 FTS5 命中 → 按引用排序。完全离线、不调外网。

综述生成 你想看某个新方向的「鸟瞰图」，但你的库里可能根本就没有这方向的论文。流程：LLM 拆分子主题 → Semantic Scholar 拉真实文献 → LLM 标注必读。

7.2 搜索召回

7.2.1 扩写查询

输入「retrieval augmented generation」会被扩写为「RAG / 检索增强 / 向量检索 / chunk + retrieval / Lewis 2020...」等十几条相关查询。这一步使用 search 任务的 LLM，目的是召回那些用了不同表述但讲同一件事的论文。

7.2.2 扩写算法细节

扩写由 query_expand 模块负责。它不会把用户原话直接扔给 S2，而是先让 LLM 把（可能是中文的）宽泛话题翻译为 2-4 个精确的英文搜索词。核心约束写在 system prompt 里：

- 只输出 2-4 个搜索词，宁少勿多——多词 OR 只会膨胀噪声。
- 选领域专家实际会输入的术语，不堆砌相邻子领域的关键词。
- 如果两个候选词会召回同一批论文（如「attosecond pulses」和「attosecond laser」），只保留更规范的那个。
- 中文输入先翻译为英文再检索。
- 去掉空泛修饰词（research / study / novel / recent 等）。

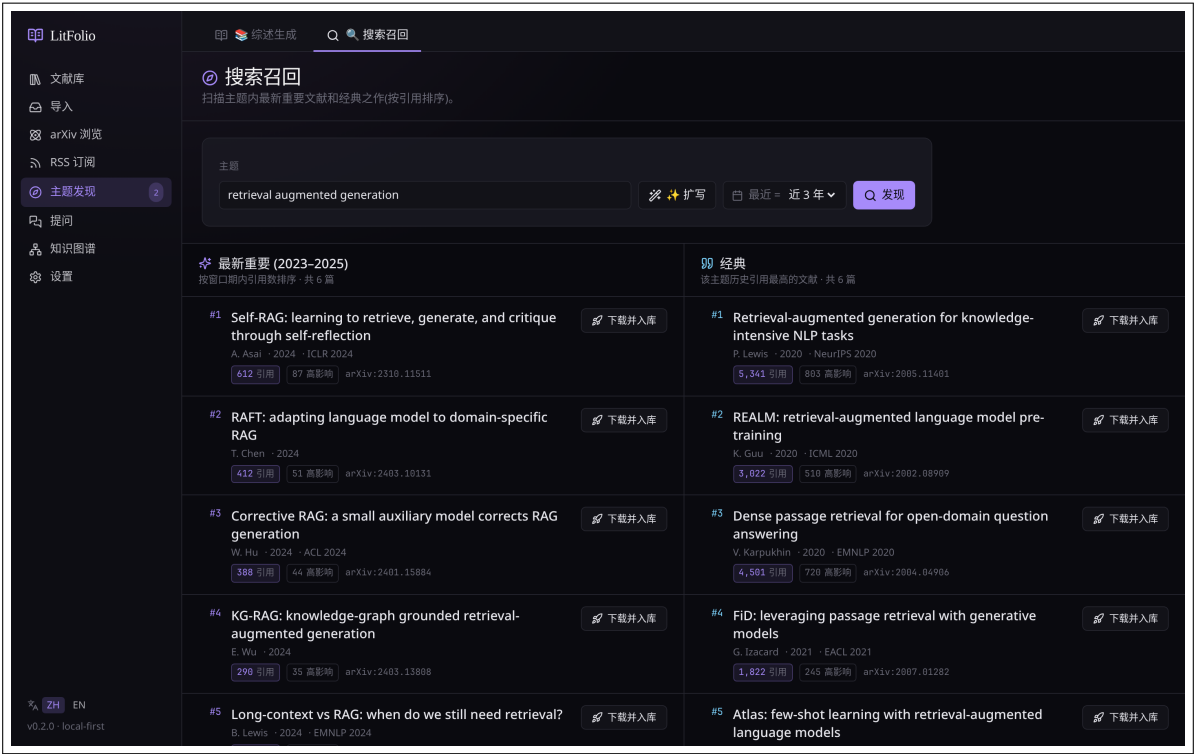


图 7.1 搜索召回 tab。输入查询后，LitFolio 会先扩写出若干相关短语，再在本地 FTS5 索引上多路召回，最后按引用数排序展示。结果可点入抽屉查看 TL;DR。

LLM 返回 JSON: {"terms": [...] }。解析器有三级容错：先尝试直接 JSON 解析，失败则提取第一个 { ... } 子串（应对 markdown fence 和闲聊包裹），最后按逗号或换行分词兜底。最多取 4 个词。

7.2.3 时间窗与排序

默认按引用数排序；右上角下拉框可改为「最新」「相关度」「必读优先」。时间窗（近 1/3/5 年）用来过滤旧论文。

7.3 综述生成

7.3.1 流水线

综述生成是一条四步流水线，每步进度通过 IPC 事件实时推送到前端。

为什么不直接让 LLM 列论文

如果让 LLM 直接输出论文列表，它会编造 DOI、作者、年份——幻觉率极高。LitFolio 的做法是把「规划」和「检索」严格分开：LLM 只负责规划（拆子领域 + 给搜索词），真正去 Semantic Scholar 拉论文的是程序代码，最后再让 LLM 对真实论文做注释。

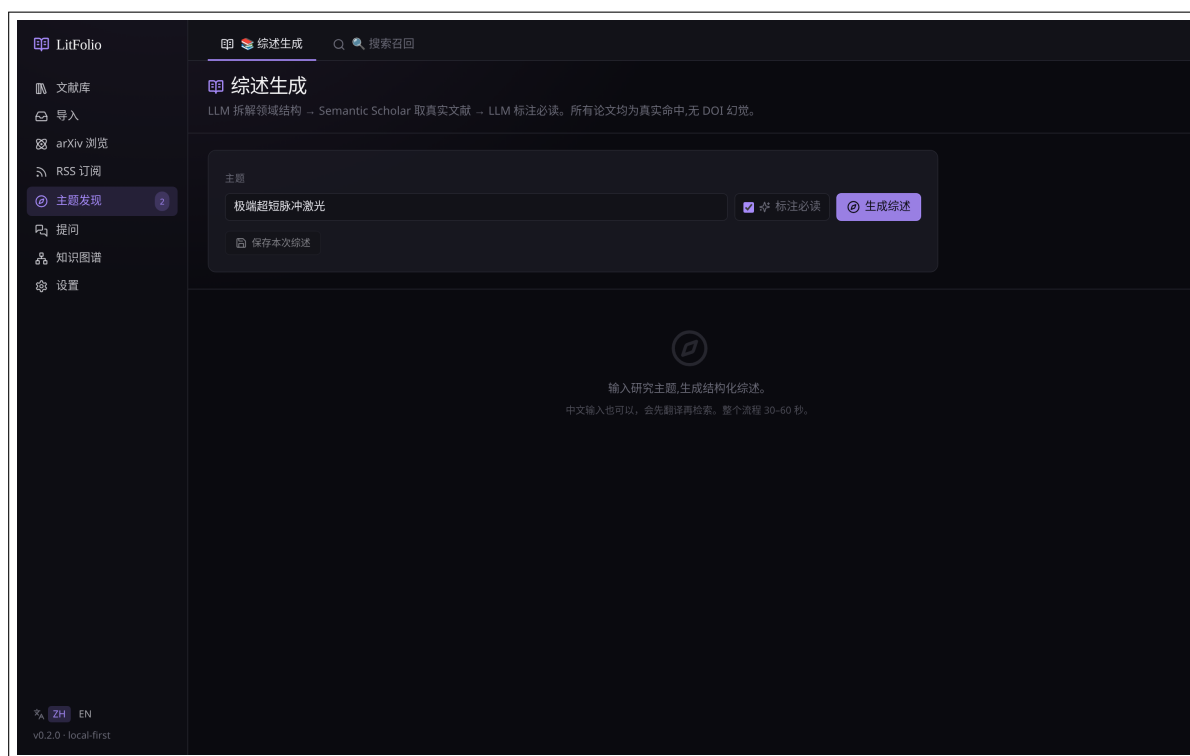


图 7.2 综述生成 tab。输入一个开放话题, LitFolio 会调用 LLM 把它拆成 3 个子领域, 然后到 Semantic Scholar 取每个子领域 4 篇代表论文, 最后 LLM 标注「必读」并给出一句话理由。

1. **planning**——把开放话题拆成 3 个子领域 (如「极端超短脉冲激光」可能拆为啁啾脉冲放大 / 谐波生成 / 光学参量过程)。
2. **grounding**——对每个子领域调 Semantic Scholar Graph API 拉 4 篇高引用论文。这里是真实文献, 不是 LLM 编的。
3. **annotating**——LLM 给每篇论文打「必读」标记 + 一句话理由 (解释为什么该方向看这篇)。
4. **done**——把整份综述保存到 SQLite topic_surveys 表, 可下次直接打开。

Phase 1: LLM 规划

LLM 收到一个 (可能是中文的) 研究话题, 输出 JSON 结构: 4-7 个子领域, 每个含名称、活跃年份范围、一段中文叙述、2-4 个英文 S2 搜索词、可选的 PI 人名提示。还有 5-10 个跨子领域的 key_PI (重要研究者)。中文输入会在 prompt 里被要求翻译为英文搜索词。

Phase 2: Semantic Scholar 接地

对每个子领域, 程序依次执行:

1. **按词检索**——每个 search_term 调一次 S2 bulk_by_citations API, 拉回按引用数排序的论文, 带年份过滤。每个词拉 $\text{topk} \times 3$ (至少 15) 条结果, 保证去重后还有足够的候选。
2. **PI 补充检索**——把 PI 人名和 search_term 拼起来搜 S2, 每个 PI 最多和 2 个搜索词组合。

3. **去重**——同一子领域内按 paperId 去重（优先 paperId，无 paperId 用 DOI），重复的保留引用数更高的版本。
4. **相关度过滤**——论文标题和摘要中必须包含至少 2 个去停用词后的搜索词 token，或者直接包含某个搜索词完整短语。S2 偶尔会召回不相关的论文，这一层过滤专门拦截它们。
5. **DOI 过滤 + 排序**——没有 DOI 的论文被丢弃（保证可追溯），剩下的按引用数降序取 top-K。

跨子领域去重采用「先到先得」策略：一篇论文如果在多个子领域都命中，只保留在先处理的那个子里。

S2 API 速率控制

LitFolio 内置了令牌桶限流：每 300 秒窗口最多 80 次请求（S2 未认证限制是 100 次/5 分钟，留 20% 余量）。用全局原子计数器实现，窗口过期自动重置。超出时返回明确错误提示「try again later」而非静默失败。综述生成时每个子领域的多个 search_term 串行调用，如果子领域多、搜索词多，可能会触发限流——这就是为什么综述生成有时需要等 30-60 秒。

Phase 3: LLM 注释（可选）

把所有子领域的真实论文列表（id + 标题 + 年份 + 摘要）交给 LLM，要求：

- 每篇写 1-2 句中文「为什么重要」。
- 在全部论文中标记 8-12 篇为「必读」，要求覆盖不同子领域（不要堆在同一个里）。

Phase 3 是可选的——如果这一步 LLM 调用失败，综述仍然可用，只是缺少注释和必读标记。

7.3.2 保存与恢复

每次生成会自动保存。「历史综述」按钮可回到旧的综述继续看（无需重新调 LLM 与 API）。

提示 中文输入完全 OK。LitFolio 会在 planning 步先把中文话题翻译成英文检索词再调 S2，所以「极端超短脉冲激光」会变成「ultrashort intense laser pulses」之类去拉文献。整个流程通常 30-60 秒。

7.4 主题提醒

主题提醒是「主题发现」的自动化延伸——设定一个研究方向后，LitFolio 会定期监控新出现的相关论文。与 RSS 订阅的区别：RSS 跟踪的是**来源**（某个 arXiv 分类或期刊），主题提醒跟踪的是**话题**（跨来源的研究方向）。

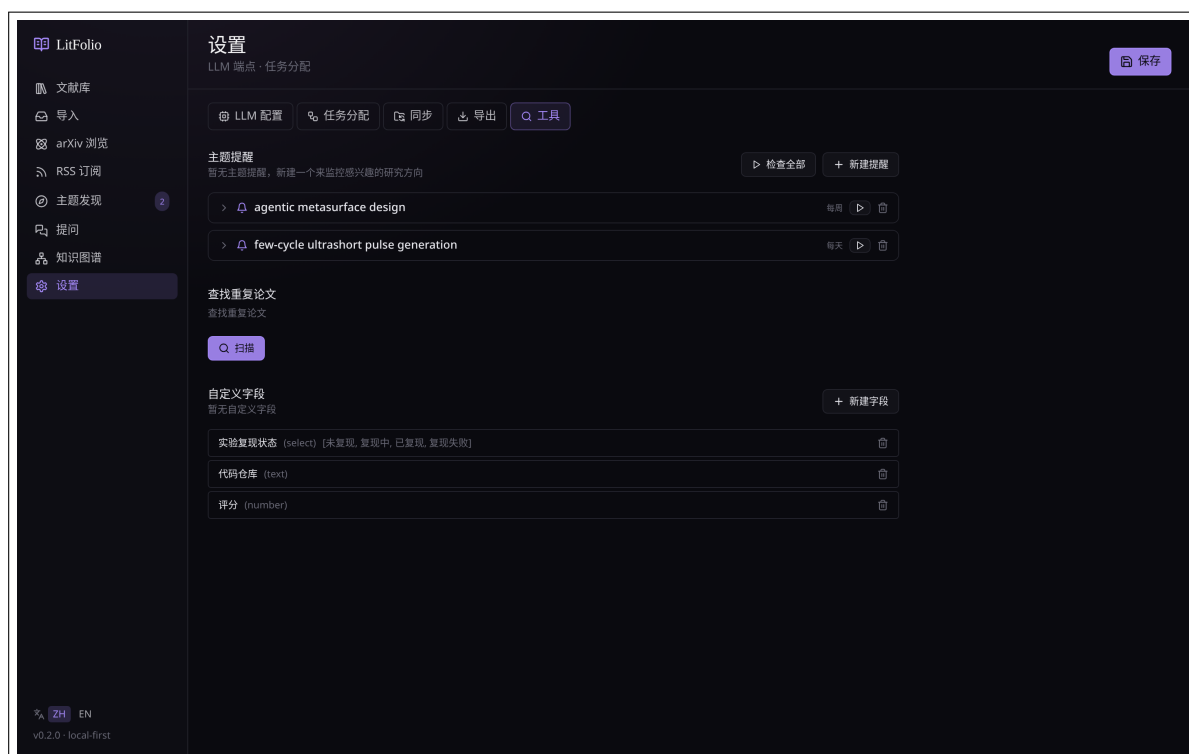


图 7.3 主题提醒管理界面。设定查询、频率和目标文件夹，系统自动运行并报告新发现的论文。

7.4.1 创建提醒

创建提醒时需要设置：

查询 研究主题的自然语言描述（与主题发现的输入方式相同），如「极端超短脉冲激光的啁啾脉冲放大技术」。

频率 每日 / 每周 / 每次启动时。

目标文件夹 可选——自动入库的论文会被放入指定文件夹。

自动入库 开启后，新发现的论文自动导入到库中。

7.4.2 提醒执行流程

提醒触发时，LitFolio 执行以下流程：

1. **LLM 规划**——将查询翻译为 2–4 个英文搜索词（与主题发现的 query_expand 使用相同的逻辑）。
2. **Semantic Scholar 检索**——每个搜索词调用 S2 API，按引用数排序拉取最新论文。
3. **去重过滤**——将检索结果与两个列表比较：
 - 已入库论文（DOI / arXiv ID 匹配）→ 排除。
 - 上次提醒已展示的论文（topic_alert_results 表）→ 排除。

只保留**既未入库也未展示过**的新论文。

4. **存储结果**——新论文写入 `topic_alert_results` 表，标记 `seen = 0`。
5. **通知**——导航栏的主题入口显示未读数量徽章。

7.4.3 自动入库

如果开启了「自动入库」，第 4 步之后会额外执行：

5. **批量入库**——对每篇新论文，调用 arXiv/DOI 入库流程拉取元数据和 PDF。
6. **放入目标文件夹**——如果指定了目标文件夹，入库的论文自动归入该文件夹。

7.4.4 管理与查看

在「设置 → 主题提醒」中管理所有提醒：

- 查看每个提醒的历史结果列表。
- 标记结果为已读 (`seen = 1`)。
- 编辑提醒的查询、频率、目标文件夹。
- 暂停或删除提醒。

注意 自动入库会消耗 LLM token（元数据翻译等）。如果你设置了多个高频提醒且开启了自动入库，建议将相关任务绑给低成本模型。

库内提问（RAG）

/ask 是把 LitFolio 当作你自己的知识库来问问题。和通用 ChatGPT 的区别：回答只基于你已经入库的论文，并且每一句结论旁边都标 [N] 引用编号。

8.1 工作机制

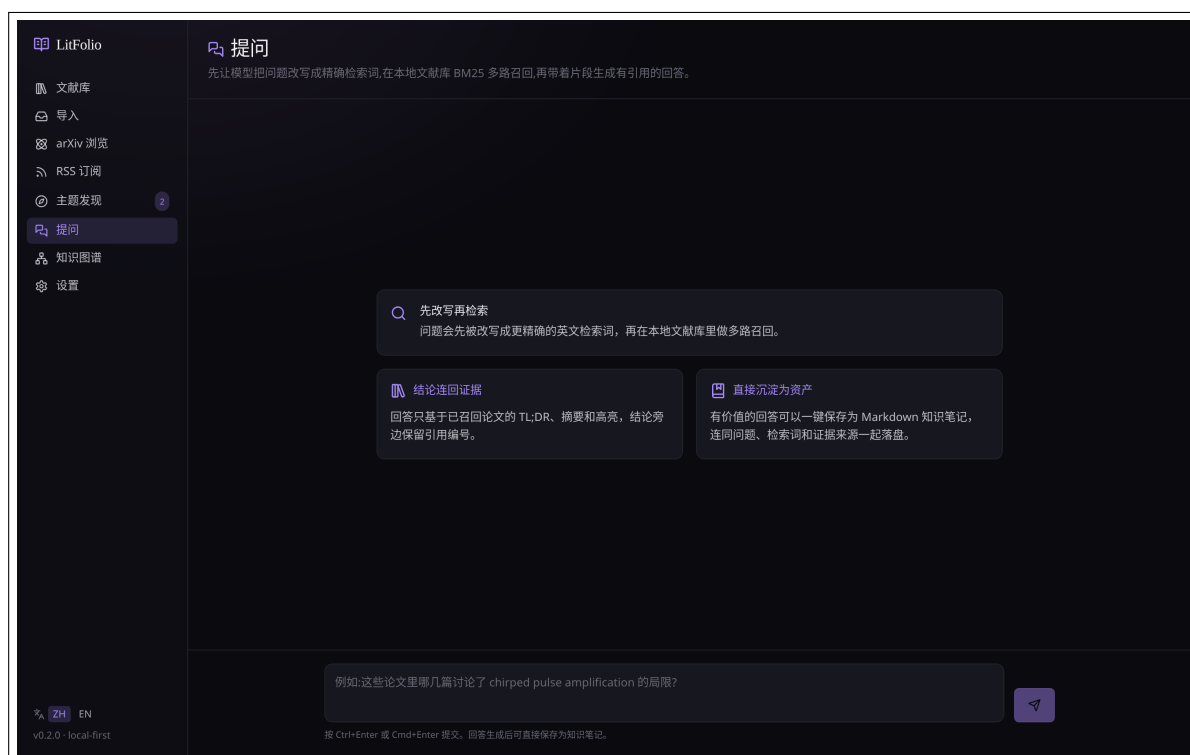


图 8.1 Ask 页初始状态。三张卡片解释工作流：1) 问题改写；2) 多路 FTS5 召回；3) LLM 回答带引用。

整个流程分四步：

1. **LLM 问题改写**——这一步是整个 RAG 流水线的关键。直接把中文问题扔进 FTS5 的 AND-of-tokens 路径基本是零召回（FTS5 unicode61 对中文按字切分，「神经网络」会变成「神 AND 经 AND 网 AND 络」）。因此 query_expand 模块先将自然语言问题改写为 2–4 个英文搜索词。改写失败时（离线、key 无效、模型未配置）不阻塞流程，降级为用原始问题直接搜索。
2. **多路 FTS5 召回 + 合并评分**——每个搜索词独立跑一次 FTS5 BM25 搜索（每词超取 limit × 3，至少 8 条），然后按 paper_id 合并。合并时的排序规则：**命中词数**（被越多搜索词命中的

论文越靠前) > 年份降序 > 入库时间降序。这样,「跨多个语义角度都相关」的论文会优先于「只在一个词上高分」的论文。如果扩写全部返回空,则用原始问题做一次 FTS5 搜索作为兜底。

3. **证据拼装**——每篇召回论文被压缩为一段不超过 1800 字符的 snippet, 按优先级拼接: TL;DR → Abstract → Problem → Method → Comparison → Limitations → Key Findings → 用户高亮 (最多 3 条, 每条 240 字符)。所有 snippet 拼在一起不超过 14000 字符, 超出则截断。
4. **LLM 回答**——system prompt 强制要求: 只基于提供的证据回答, 每个事实标注 [N] 引用编号, 证据不足时明说「找不到相关文献」而不是猜测, 用中文回复。如果检索结果为空, 直接返回固定提示文案而不调 LLM。

8.2 多轮对话界面

Ask 页已升级为聊天式对话界面, 支持连续追问。每轮对话显示用户问题和 AI 回答, 带有独立的头像标识 (用户蓝色、AI 绿色)。

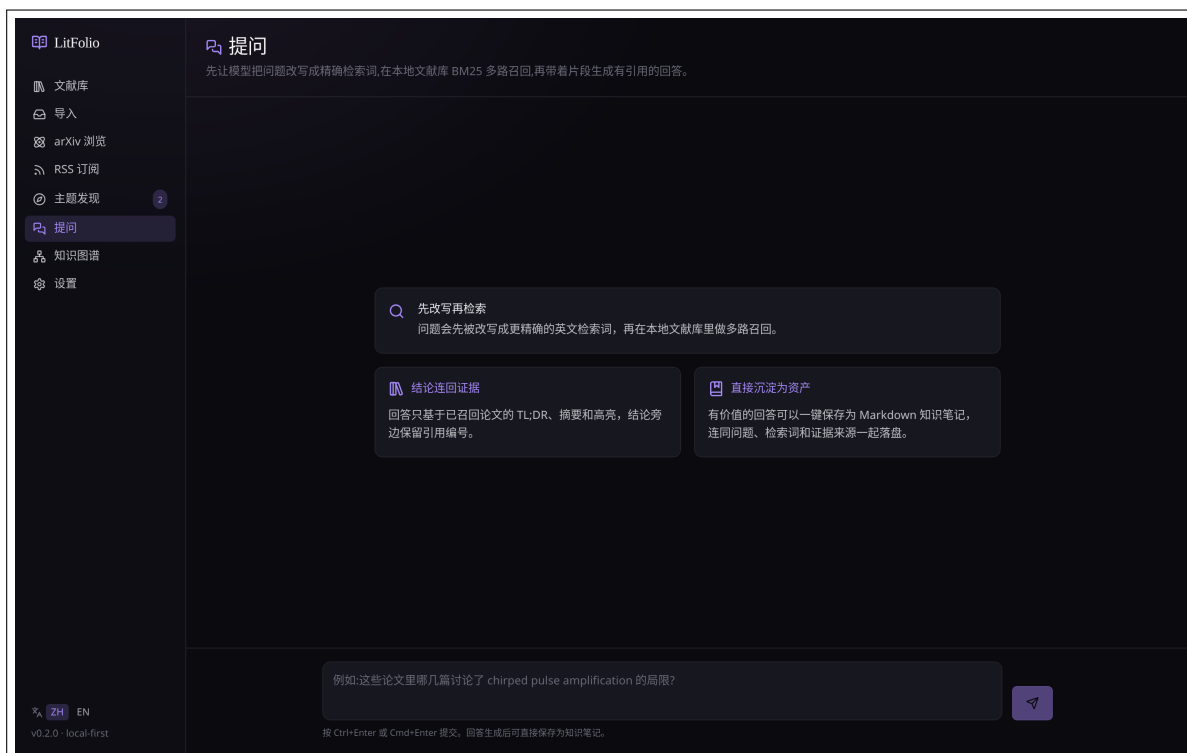


图 8.2 Ask 多轮对话。用户和 AI 交替显示消息气泡, 每条回答附带来源引用和保存按钮。

8.2.1 对话状态模型

前端维护一个 ConversationTurn[] 数组, 每个元素包含:

- role——"user" 或 "assistant"。

- **content**——消息文本。
- **result**——仅 assistant 角色有, 包含完整的 `AskLibraryResult` (`answer`、`sources`、`model`、`prompt_tokens`、`completion_tokens`、`terms`)。

对话历史**仅在当前会话中保持**——关闭页面或切换路由后清空。这与单轮模式的行为一致(之前的单轮结果也不跨会话持久化)。

8.2.2 多轮 RAG 流程

每轮追问的处理流程与单轮完全相同(问题改写 → FTS5 召回 → 证据拼装 → LLM 回答), 区别在于 LLM 的消息数组构建方式:

1. **System prompt**——与单轮相同的引用严格系统提示。
2. **历史消息**——将之前的对话历史(所有 user/assistant 消息对)作为 `ChatMessage` 数组插入到 `system` 和当前问题之间。
3. **当前问题 + 证据**——当前追问的文本加上本轮新检索到的 `Sources` 块。

这意味着:

- **上下文传递**——LLM 能看到之前的问答, 因此代词和指代可以被正确解析。例如「它用了什么方法?」中的「它」可以指向上一轮讨论的论文。
- **每轮独立检索**——每轮追问都会重新做问题改写和 FTS5 召回, 而不是只用第一轮的证据。这保证了追问可以覆盖新的语义角度。
- **引用独立**——每条回答的 [N] 编号只对应本轮的 `Sources`, 不会与前几轮的编号混淆。每条回答下方独立展示来源卡片。

8.2.3 对话历史传递的细节

发给后端的 `conversation_history` 是一个 `Vec<ConversationMessage>`, 每个元素有 `role` 和 `content`。后端将其转换为 `ChatMessage` 数组传给 LLM。注意: 历史消息中**不包含** `Sources` 块——`Sources` 只附加在当前追问的 `user` 消息中。这样做的原因是:

- **避免 token 爆炸**——如果每轮都保留完整的 `Sources` 块, 5 轮对话后 `context` 可能超过模型窗口。
- LLM 只需要当前轮的证据来回答当前问题——之前的证据已经体现在之前的 `assistant` 回答中。

8.2.4 UI 交互

- 输入框在页面底部, `Ctrl+Enter` / `Cmd+Enter` 发送。
- 占位文字在有对话历史时变为「继续追问...」。

- 新消息发送后自动滚动到底部 (`scrollIntoView + smooth behavior`)。
- 顶部「清除对话」按钮重置整个对话历史和输入框。
- 加载中显示 AI 头像 + 旋转图标 + 「正在检索…」文字。
- 错误信息以红色文字显示在输入框下方。

提示 多轮对话的质量取决于上下文窗口大小。如果你的 LLM 配置的 `context window` 较小 (如 4K tokens), 5 轮以上的对话可能会截断历史。建议使用 8K+ 窗口的模型。

8.3 保存为知识笔记

每条回答下方有「保存为笔记」。点击会生成一份独立的 Markdown 文档(位于 `notes/ask/<timestamp>.md`), 完整保留**问题、检索词、答案、证据来源**。WebDAV 同步时这份笔记会一起被推送。

提示 RAG 的回答质量直接取决于你库的覆盖度——库里没有相关论文, 回答会诚实地说「找不到明显相关的文献」。这是**特性**: 不会编造没读过的文献。多轮对话时, 如果第一轮找不到文献, 后续追问也不会凭空编造。

知识图谱

/graph 路由是 LitFolio 的文献关系可视化中心。它把论文之间的引用、方法继承、对比等关系，以及论文与概念术语之间的关联，汇聚为一张可交互的知识图谱。

9.1 网络图

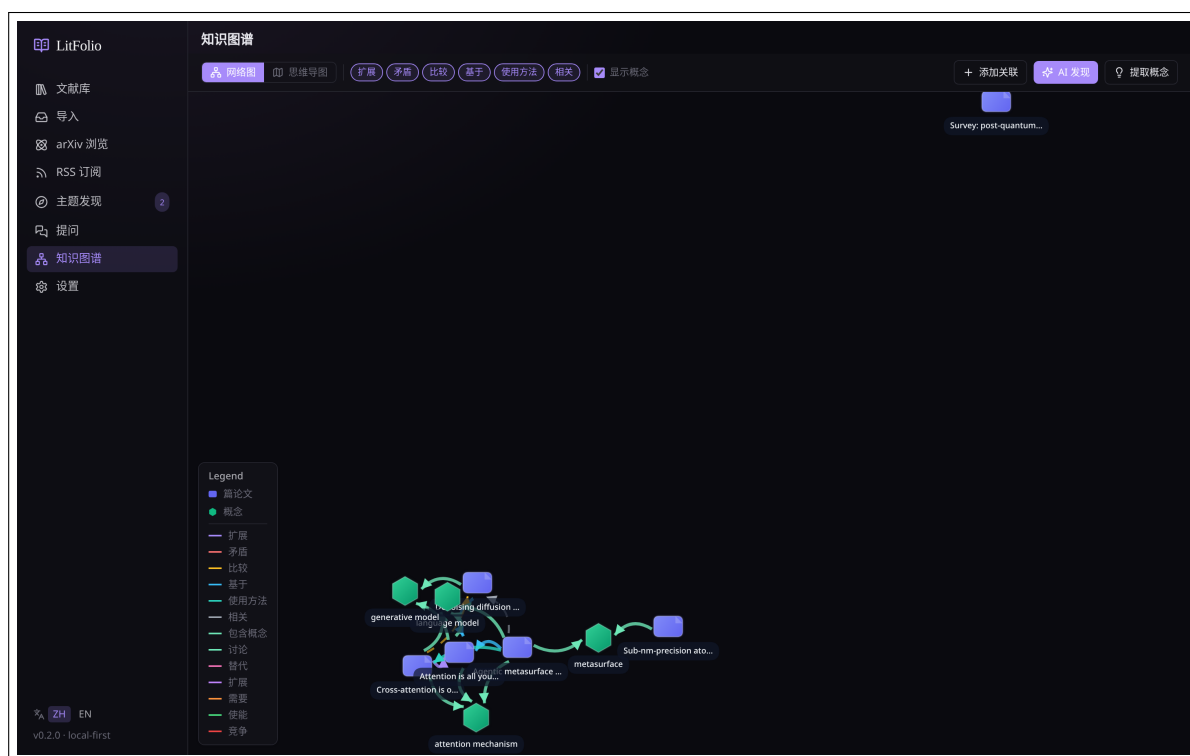


图 9.1 知识图谱——网络视图。蓝色节点为论文，绿色节点为概念；连线颜色表示关系类型，虚线为 AI 发现的潜在关联。

网络图是默认视图，使用力导向布局自动排布节点：

- **论文节点**（蓝色）——大小固定，悬停显示标题与年份。
- **概念节点**（绿色）——来自术语表中被两篇以上论文共享的 `normalized_term`，节点旁标注关联论文数。
- **连线**——颜色按关系类型区分（见图例），实线为用户手动创建，虚线为 AI 发现。

点击任意节点，右侧边栏会展示该节点的详细信息和所有关联节点列表。

9.2 思维导图

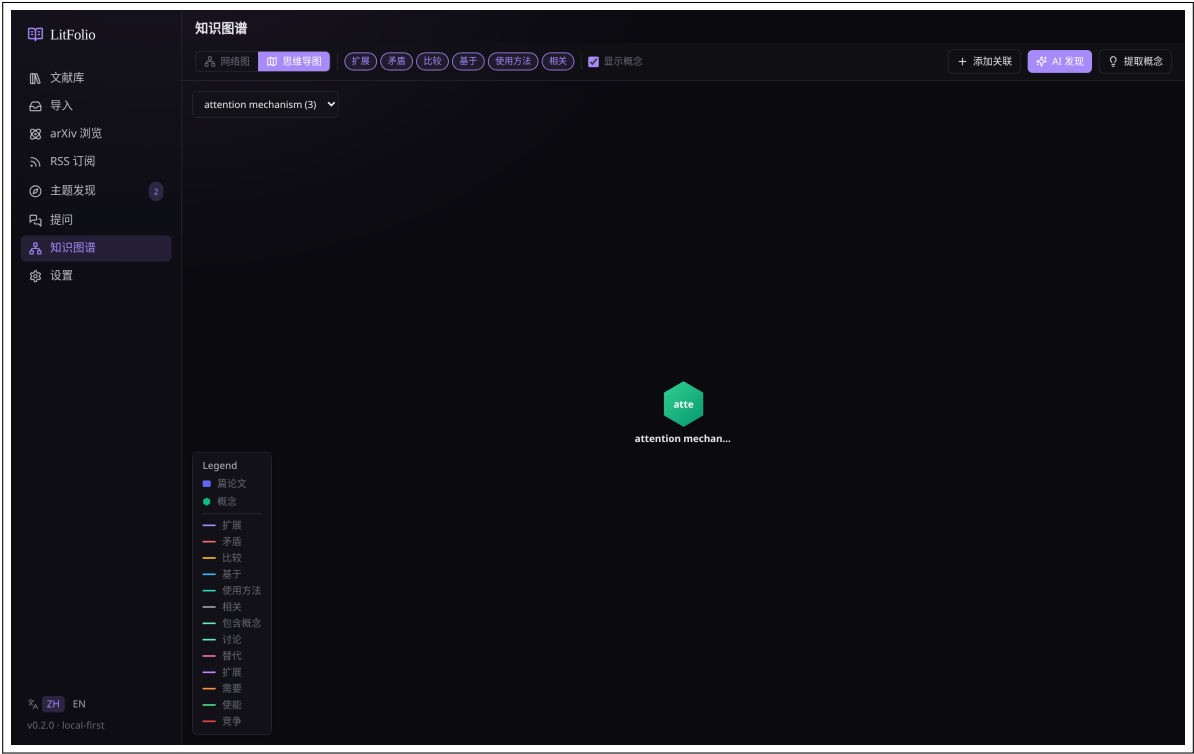


图 9.2 知识图谱——思维导图视图。以概念为中心向外辐射，第一环为关联论文，第二环为这些论文的链接论文。

思维导图以某个概念术语为根节点，按环形向外辐射：

- **第一环**——通过术语表直接关联到该概念的论文。
- **第二环**——与第一环论文存在 `paper_links` 关系的其他论文。
- 点击概念节点上的「定位」按钮可切换到以该概念为中心的思维导图。

9.3 手动创建关系

工具栏上的「添加链接」按钮打开创建对话框。你需要选择：

1. **来源论文**——默认为当前选中的论文节点。
2. **目标论文**——从下拉列表中选择。
3. **关系类型**——六种预定义类型：`extends`（扩展）、`contradicts`（矛盾）、`compares`（对比）、`builds_on`（基于）、`uses_method`（使用方法）、`related`（相关）。
4. **说明（可选）**——一段简短的文字描述两篇论文的具体关联。

用户创建的关系置信度固定为 1.0，以实线显示。

9.4 引用网络

除了用户手动创建和 AI 发现的关系，LitFolio 还能从 Semantic Scholar 拉取论文的学术引用关系——展示「学术界认为这篇论文与谁相关」。在论文抽屉或知识图谱页面点「引用」按钮。

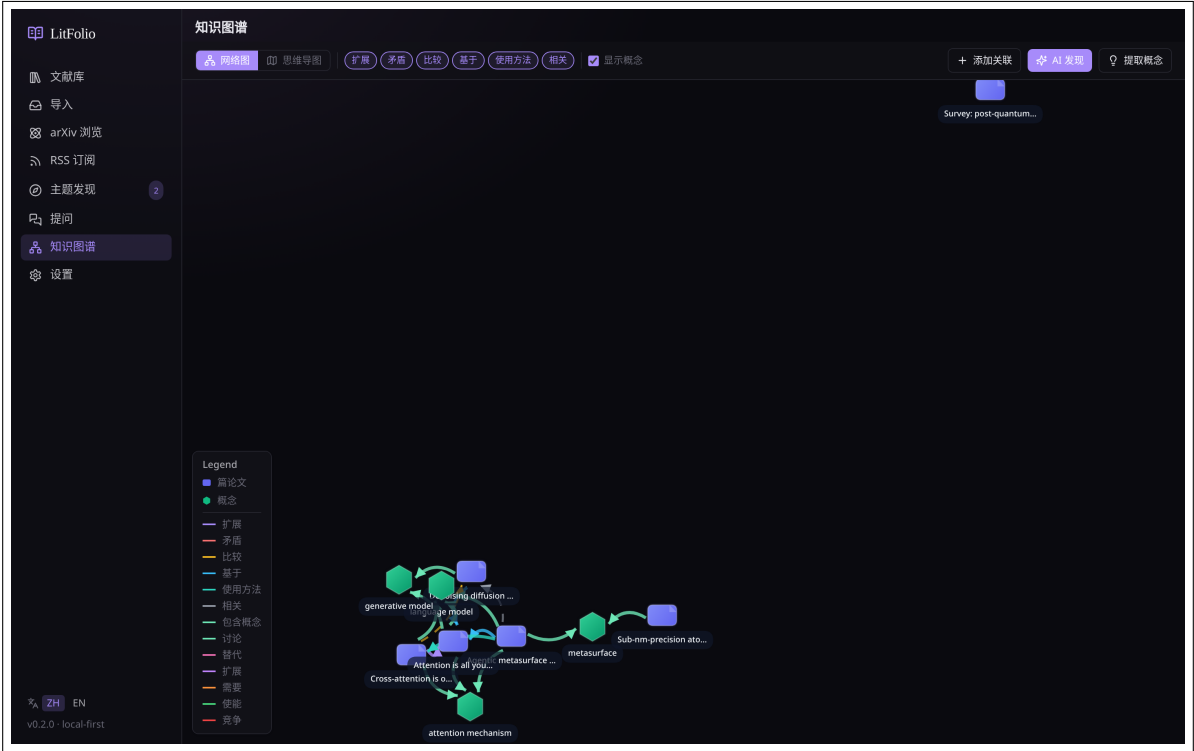


图 9.3 引用网络视图。中心论文左侧为它引用的论文，右侧为引用它的论文，已入库论文以绿色边框标识。

9.4.1 数据获取流程

引用数据通过 Semantic Scholar Graph API 获取：

- 1. **论文匹配**——用论文的 DOI 或标题向 S2 发起匹配请求。优先使用 DOI（精确匹配）；无 DOI 时用标题模糊匹配，S2 返回最相关的 paperId。
- 2. **拉取引用关系**——调用 GET /paper/{id}?fields=references,citations,返回两个数组:references（该论文引用了谁）和 citations（谁引用了该论文）。每个元素包含 title、authors、year、venue、externalIds。
- 3. **缓存**——结果写入 paper_citations 表，每条记录包含方向（references / citations）和 fetched_at 时间戳。再次查看时直接读缓存，不重复调 API。

9.4.2 树形展示

引用网络以树形布局展示：

- **中心节点**——当前论文。
- **左向分支**——References（该论文引用了哪些论文），最多 2 层深度（第一层是直接引用，第二层是这些引用论文各自的引用）。
- **右向分支**——Citations（哪些论文引用了该论文），同样最多 2 层。
- **入库标识**——已在你库中的论文以绿色边框高亮，悬停显示 TL;DR（如有）。
- **未入库论文**——显示标题、作者、年份；点击可查看摘要详情，并有「入库」按钮。

9.4.3 与知识图谱的关系

引用网络和知识图谱是互补的两个维度：

引用网络 来自学术社区的客观数据——谁引用了谁，不依赖你的判断。

知识图谱 来自你的主观判断和 AI 推断——论文之间的方法继承、矛盾、对比等关系。

两者可以交叉验证：如果知识图谱中 A 延展了 B，而引用网络中 A 确实引用了 B，这条关系的可信度就更高。

注意 Semantic Scholar 的引用数据覆盖率并非 100%——非常新的论文（发表不到 1 周）或小众期刊的论文可能没有引用数据。此时 API 返回空数组，界面会显示「暂无引用数据」。

9.5 概念图谱

概念图谱是知识图谱的进阶维度——它展示的不是论文之间的关系，而是**方法论概念之间的演化关系**。点击工具栏上的「提取概念」按钮启动。

9.5.1 概念提取流水线

点击「提取概念」后，LitFolio 逐篇论文执行以下流程：

1. **文本准备**——取论文的 TL;DR + 摘要 + 深读四段（如有），拼接为不超过 3000 字符的输入文本。
2. **LLM 提取**——将文本交给 LLM，system prompt 要求返回 JSON 数组，每个元素包含：
 - name——方法论概念名称（如 Transformer、LoRA、Self-Attention），要求规范化命名（首字母大写、不用缩写除非缩写更通用）。
 - description——一句话中文描述该概念。



图 9.4 概念图谱。概念节点（绿色）之间用不同颜色的连线表示关系类型：替代（粉色）、扩展（紫色）、需要（橙色）、赋能（绿色）、竞争（红色）。

- **relations**——与其他概念的关系列表，每条含 **target**（目标概念名）、**relation**（关系类型）、**snippet**（原文证据片段）。
3. **JSON 解析**——三级容错（直接解析 → 提取 [...] 子串 → 空数组兜底），与 TL;DR 的解析策略一致。
 4. **去重与规范化**——将提取到的概念名转小写做去重比较。如果库中已有同名概念（忽略大小写），复用已有概念的 ID；否则创建新概念。
 5. **关系创建**——对每条关系，查找目标概念（同名匹配），如果目标概念不存在则先创建。然后在 **concept_relations** 表中创建边，记录 **evidence_paper_id** 和 **snippet**。
 6. **论文-概念关联**——在 **paper_concepts** 表中创建 **discusses** 边，默认 **relevance = 1.0**。

9.5.2 概念关系类型

五种预定义关系类型，每种对应不同的学术语义：

类型	含义	典型示例
replaces	新方法取代旧方法	Transformer replaces RNN
extends	在已有方法上改进	LoRA extends Fine-tuning

类型	含义	典型示例
requires	依赖另一方法	Attention requires Embedding
enables	使另一方法成为可能	GPU enables Deep Learning
competes_with	同时期替代方案	BERT competes_with GPT

9.5.3 图谱可视化集成

提取完成后，概念节点自动出现在知识图谱中：

- **网络图**——概念节点为绿色，大小固定。概念间的关系用不同颜色连线：**replaces** 粉色、**extends** 紫色、**requires** 橙色、**enables** 绿色、**competes_with** 红色。论文与概念之间用 **discusses** 边（浅绿色）连接。
- **思维导图**——以概念为根节点向外辐射。第一环为与该概念直接关联的论文（通过 **paper_concepts**），第二环为这些论文的链接论文（通过 **paper_links**）。
- **图例**——左下角图例面板标注了所有关系类型的连线颜色和样式。

9.5.4 手动管理

你可以对 AI 提取的结果进行手动修正：

- **创建概念**——手动添加 AI 遗漏的概念。
- **编辑概念**——修改名称或描述。
- **删除概念**——删除后，关联的 **paper_concepts** 和 **concept_relations** 记录会被级联删除。
- **创建/删除关系**——手动添加或移除概念间的关系。
- **关联/取消关联论文**——管理概念与论文之间的 **discusses** 边。

提示 概念提取的质量取决于论文的深读质量。如果论文只有 TL;DR 而没有深读四段，LLM 能获取的上下文较少，提取的概念可能不够精确。建议先对论文运行深读，再做概念提取。

9.6 AI 发现关系

点击工具栏上的「AI 发现」按钮，LitFolio 会：

1. 收集库内所有论文的元数据（标题、年份、TL;DR、方法、关键发现）。
2. 按共享术语分批（每批最多 12 篇），交给 LLM 分析潜在关联。
3. 解析 LLM 返回的 JSON，为每对论文生成带置信度和说明的关系建议。

AI 发现的关系以虚线显示，置信度低于 1.0。在阅读器的术语面板中，AI 关系旁会标注置信百分比。你可以：

- **接受**——将 AI 关系提升为用户关系（置信度变为 1.0）。
- **拒绝**——删除该关系建议。

9.7 筛选与图例

工具栏提供两组筛选：

- **关系类型**——按类型标签过滤显示哪些连线。
- **包含概念**——开关概念节点的显示。关闭后只显示论文间的直接关系。

左下角的图例面板标注了节点颜色和每种关系类型对应的连线样式。

9.8 阅读器集成

在阅读器的术语面板（第 4 章）底部，会显示当前论文的所有关联文献。每条关联标注了关系类型和来源（用户/AI），点击即可跳转到对应论文的阅读器视图。

提示 知识图谱的数据完全来自本地 SQLite 数据库（`paper_links` 表和 `paper_terms` 表的交集）。WebDAV 同步时关系数据会随库一起推送。

相似论文推荐

读完一篇论文后的自然问题是「还有什么值得读?」。相似论文推荐用 Semantic Scholar 的推荐 API 自动找到相关文献。

10.1 使用方式

在论文抽屉或阅读器顶部点「相似论文」按钮。LitFolio 会用论文的 DOI 或标题向 Semantic Scholar 发起推荐请求。

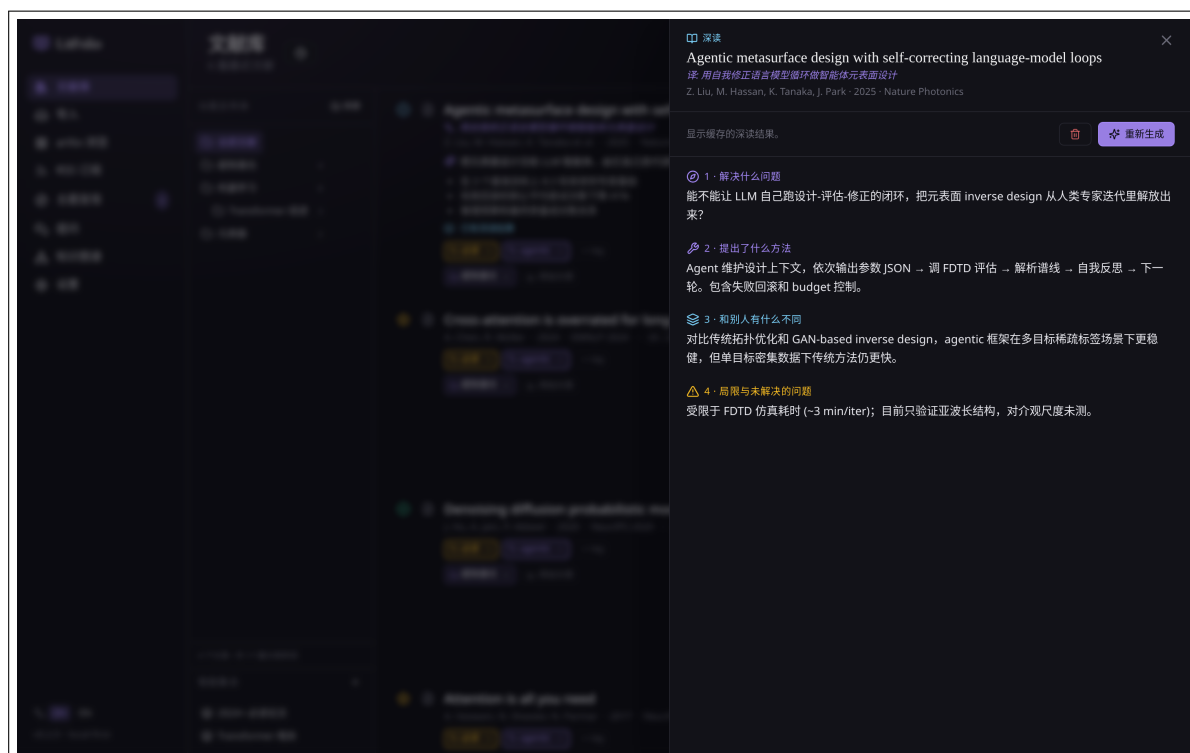


图 10.1 相似论文推荐面板。结果卡片展示标题、作者、年份、摘要片段和相关度评分。

10.1.1 推荐获取流程

1. 论文匹配——用 DOI（优先）或标题向 Semantic Scholar 匹配 paperId。与引用网络使用相同的匹配逻辑。

2. **调用推荐 API**——请求 GET /paper/{id}/recommendations, 返回按相关度排序的论文列表。每条包含 title、authors、year、abstract、citationCount。
3. **库内过滤**——遍历推荐结果, 用 DOI 和 arXiv ID 检查是否已在库中。已入库的论文从结果中移除。
4. **缓存**——过滤后的结果写入 recommendation_cache 表, 附带 fetched_at 时间戳。7 天内再次查看直接读缓存。

10.1.2 结果展示

每张结果卡片包含:

- 标题、作者 (前 3 位 + et al.)、年份。
- 摘要片段 (截取前 200 字符)。
- 引用数 (如 S2 返回了该字段)。
- 「入库」按钮——点击后拉取完整元数据并尝试下载 PDF。

10.1.3 边界情况

- **无 DOI 的论文**——标题模糊匹配可能返回错误的 paperId, 导致推荐结果不相关。此时界面会显示「匹配精度较低, 结果仅供参考」。
- **非常新的论文**——S2 的推荐模型需要论文有一定引用积累才能生成推荐。发表不到 1 个月的论文通常没有推荐结果。
- **小众领域**——S2 数据覆盖率并非 100%, 某些非英文或非主流会议的论文可能匹配不到。

提示 如果推荐结果为空, 可以尝试用「主题发现」(第 7 章) 的手动搜索功能, 或在知识图谱中查看 AI 发现的关联论文。

文献综述草稿生成

当你为某个研究方向收集了 20–50 篇论文后，第一稿文献综述是最难写的。LitFolio 的 AI 综述生成能帮你搭好骨架。

11.1 使用方式

在文献库中多选论文（勾选模式），工具栏点「生成综述」。也可以在文件夹右键菜单中选择「生成综述」。

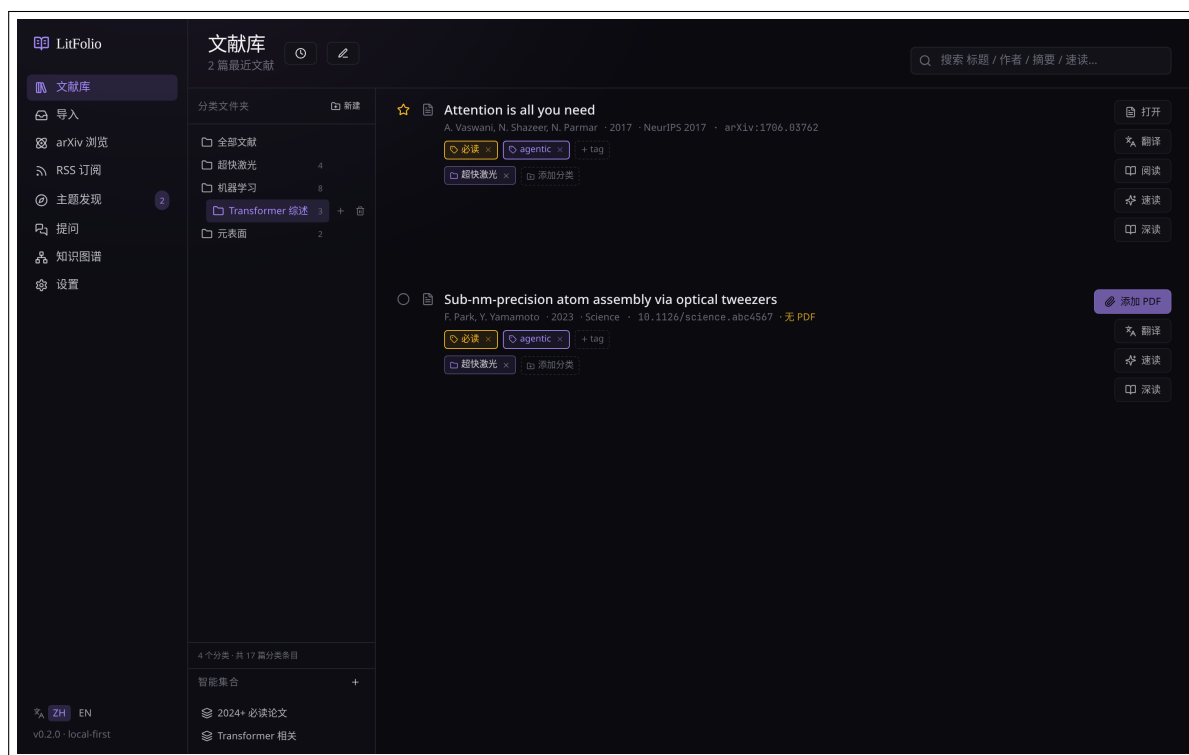


图 11.1 生成的文献综述草稿。按主题分节，每节引用相关论文，末尾列出研究空白。

11.1.1 生成流水线

综述生成是一条两步流水线：

Step 1: 分组

将选中的论文交给 LLM，要求按主题/方法/应用领域分组。LLM 返回 JSON：

```
{
  "groups": [
    {
      "name": "注意力机制变体",
      "paper_ids": ["p1", "p3", "p7"],
      "rationale": "这些论文都在改进注意力计算效率"
    },
    {
      "name": "位置编码方法",
      "paper_ids": ["p2", "p5"],
      "rationale": "解决 Transformer 的位置感知问题"
    }
  ]
}
```

分组策略可选：method（按方法，默认）、year（按年代）、application（按应用领域）。不同策略会产生不同的分组结果。

Step 2: 撰写

将分组结果和每篇论文的 TL;DR + 深读四段 + 摘要拼接为 prompt，要求 LLM 撰写综述。System prompt 约束：

- 每个分组对应一个二级章节。
- 每篇论文在正文中至少被引用一次，引用格式为 [Author, Year]。
- 引言概述研究方向的背景和综述范围。
- 末尾有一个「研究空白」章节，指出当前文献中尚未解决的问题。
- 结论总结主要发现和未来方向。
- 语言为学术中文，风格与正式综述论文一致。
- 不编造未提供的论文或数据。

11.1.2 输入材料

每篇论文提供给 LLM 的材料包括（按优先级，总 token 不超过窗口限制）：

1. TL;DR + 关键发现（约 100 tokens/篇）

2. 深读四段：Problem / Method / Comparison / Limitations (约 400 tokens/篇)

3. 摘要 (约 200 tokens/篇)

20 篇论文的输入约 14000 tokens, 加上 system prompt 和输出空间, 需要至少 16K context window 的模型。

11.2 输出与编辑

- 输出为 Markdown 格式, 2000–5000 词 (20 篇论文时)。
- 可在界面上直接编辑, 纠正 AI 的概括或补充你的观点。
- 可保存为笔记文件 (`notes/review/<timestamp>.md`)。
- 可重新生成——选择不同的分组策略 (按方法 / 按年份 / 按应用领域)。

注意 综述草稿是起点, 不是成品。AI 可能遗漏论文间的细微差异或过度概括。请务必人工审读并补充你自己的分析。尤其注意: AI 可能会过度强调某几篇论文而忽略其他——检查「研究空白」章节是否覆盖了你认为重要的方向。

第三部分

高级工具与运维

多论文对比

选中 3–5 篇同主题的论文后，AI 可以生成一份结构化对比矩阵，帮你快速梳理它们的方法、数据集、指标和结论差异。

12.1 使用方式

在文献库中勾选论文（至少 2 篇），工具栏点「对比」。LitFolio 会调用 LLM 对每篇论文的 TL;DR + 深读内容进行结构化分析。

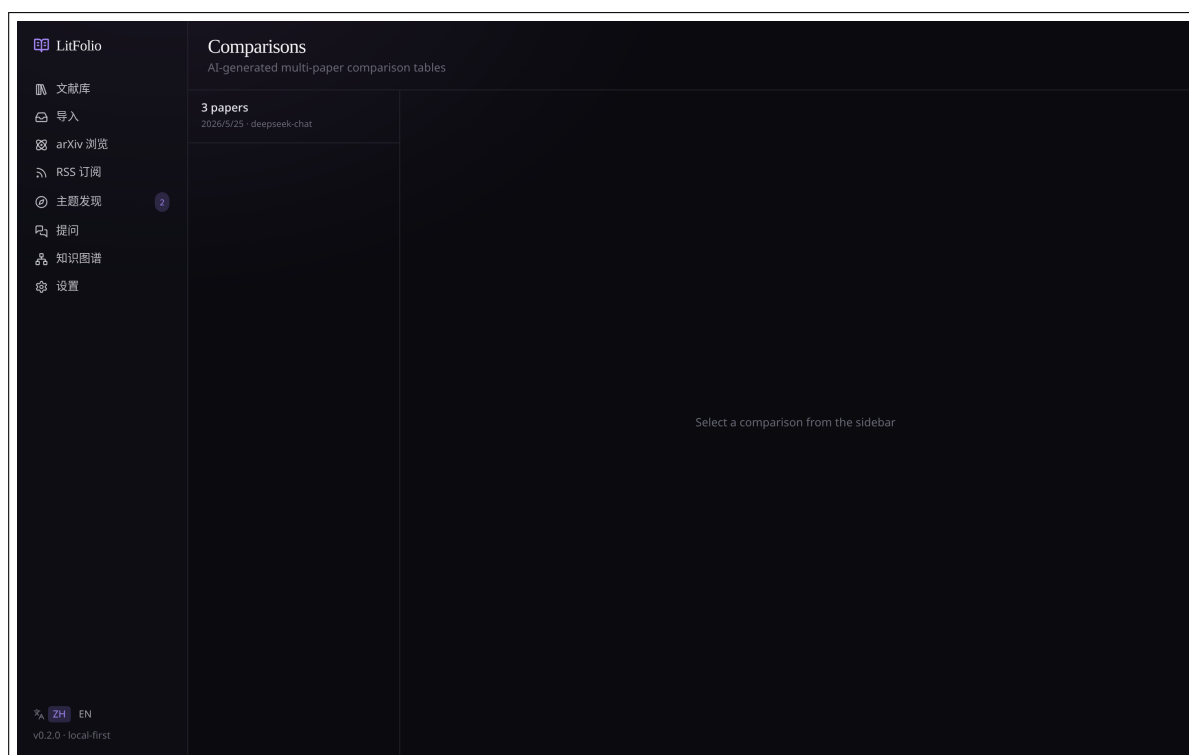


图 12.1 多论文对比页。表格按 Problem / Method / Dataset / Key Metric / Limitation / Conclusion 六列组织，单元格可直接编辑。

12.1.1 对比生成流程

1. **材料收集**——对每篇选中论文，查询其 TL;DR、关键发现、深读四段（Problem / Method / Comparison / Limitations）、摘要。如果某篇论文没有深读结果，只用 TL;DR + 摘要。

2. **Prompt 构建**——将所有论文的材料按固定格式拼接, 每篇用 `=== PAPER: <title> ===` 分隔。System prompt 要求 LLM 返回 JSON 数组, 每个元素对应一篇论文, 包含六列内容。
3. **JSON 解析**——三级容错解析 (与 TL;DR 一致)。如果解析失败, 将 LLM 的原始文本按表格格式尝试提取。
4. **存储**——结果写入 `paper_comparisons` 表, `paper_ids` 字段存储为 JSON 数组, `content` 字段存储为 Markdown 表格文本。

12.1.2 对比表格结构

对比结果以表格呈现, 列包括:

列名	内容
Paper	论文标题
Problem	解决的问题 (2-3 句)
Method	核心方法 (重点描述创新点)
Dataset	使用的数据集和规模
Key Metric	主要指标名称和数值 (如 BLEU 41.2、F1 0.89)
Limitation	局限性 (1-2 句)
Conclusion	结论 (1 句)

12.1.3 编辑与保存

- 表格支持**直接编辑**——点击单元格即可修改 AI 的输出。编辑后的内容自动保存到数据库。
- 对比结果跨会话持久化——下次打开无需重新生成。
- 「重新生成」按钮用最新的论文内容重跑 AI (覆盖已有结果)。
- 5 篇论文的对比通常在 30 秒内完成。

提示 对比质量取决于论文的深读质量。如果某篇论文还没有运行过深读, 对比时只会用到 TL;DR 和摘要, 信息量会少一些。建议先对所有待对比论文运行深读。

Markdown 导出

LitFolio 的笔记和高亮可以导出为结构化 Markdown 文件，兼容 Obsidian、Logseq 和通用 Markdown 工作流。

13.1 导出内容

每篇论文生成一个 .md 文件，文件名格式：< 第一作者 >_< 年份 >_< 标题首词 >.md。文件名中的特殊字符被替换为下划线。

13.1.1 文件结构

```
---
title: "Attention Is All You Need"
authors: ["Vaswani, Ashish", "Shazeer, Noam"]
year: 2017
venue: "NeurIPS"
doi: "10.48550/arXiv.1706.03762"
arxiv_id: "1706.03762"
tags: ["transformer", "attention"]
folders: ["ML/Transformer"]
read_status: "read"
bibtex: "@inproceedings{vaswani2017attention, ...}"
custom_project: "thesis-ch3"
custom_priority: "high"
---
```

Notes

自由文本笔记内容（来自 note.md）。

Structured Notes

Problem

解决什么问题...

Method

提出了什么方法...

Key Numbers

关键实验数据...

Limitations

局限与未解决的问题...

Thoughts

你自己的想法...

Highlights

Key Finding

- **[p.3]** The self-attention mechanism connects all positions with a constant number of operations.
 - > 用户评论（如有）

Method

- **[p.5]** Multi-head attention allows the model to attend to information from different representation subspaces at different positions.

Terms

- **Transformer**: 基于自注意力机制的序列到序列模型，摒弃了循环和卷积结构。
 - 证据: "We propose a new simple network architecture, the Transformer."
 - 跨论文复用: 3

13.1.2 Frontmatter 字段

YAML frontmatter 包含论文的所有元数据字段。自定义元数据字段以 `custom_< 字段名 >` 的格式写入。标签和文件夹以数组形式输出，方便 Obsidian 的 Dataview 插件查询。

13.1.3 高亮按标签分组

高亮输出时按标注颜色标签分组（参见 4.6 节）。每个标签作为三级标题，其下的高亮按页码排序。每条高亮包含：

- 页码和颜色标签。
- 高亮文本（加粗）。
- 用户评论（如有的话，以引用块形式呈现）。

13.1.4 术语与双向链接

术语输出时，每个术语名称用 `[[术语名]]` 双方括号包裹，利用 Obsidian 的 `wikilink` 语法实现图谱集成。如果同一篇论文的笔记中引用了某个术语，Obsidian 会自动建立双向链接。

13.2 增量导出算法

增量导出通过 `papers.last_exported_at` 字段追踪：

1. 查询 `SELECT id FROM papers WHERE last_exported_at IS NULL OR updated_at > last_exported_at`。
2. 对每篇论文，比较 `updated_at`（最后元数据/笔记/高亮变更时间）和 `last_exported_at`（最后导出时间）。
3. 只导出 `updated_at > last_exported_at` 的论文。
4. 导出成功后更新 `last_exported_at = 当前时间戳`。

这保证了批量导出时不会重复写入未变更的文件。

13.3 配置与触发

在「设置 → 导出」中配置：

- **导出目录**——如 `~/ObsidianVault/LitFolio/`。
- **增量导出**——默认开启。关闭后每次导出全部论文。
- **自动导出**——开启后，笔记或高亮保存时自动触发导出（防抖 5 秒，避免频繁写入）。

也可以手动点「立即导出」批量导出全部论文。500 篇论文的完整导出通常在 10 秒内完成。

提示 Obsidian 会自动解析 YAML frontmatter，你可以在 Obsidian 的属性视图中按 `year`、`read_status`、`tags` 等字段筛选和排序论文。配合 `Dataview` 插件可以创建动态查询视图，如「所有 2024 年以后的未读论文」。

命令面板

命令面板是 LitFolio 的键盘驱动快捷操作中心。按 **Ctrl+K**（macOS: **Cmd+K**）在任意页面打开。

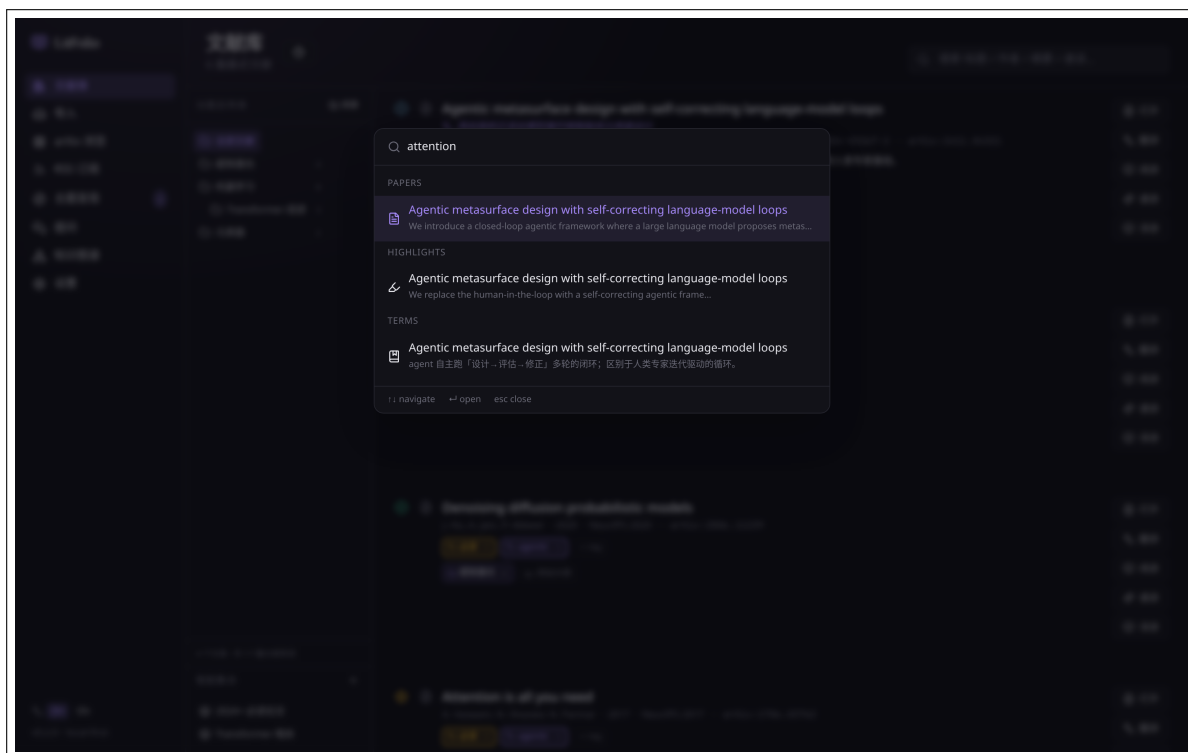


图 14.1 命令面板。顶部输入框支持模糊搜索，结果按导航/论文/操作三类分组。

14.1 三种模式

命令面板同时索引三类数据源，输入时实时过滤：

导航 页面路由名称（library、reader、graph、settings、browse、feeds、topic、ask、import、compare）。输入后跳转到对应页面。

搜索 论文标题和作者名。从数据库查询最近 20 篇论文的标题，与用户输入做模糊匹配。选中后在阅读器中打开。

命令 可执行动作，如 export bibtex、generate terms、ai discover、extract concepts、find

duplicates。选中后直接执行对应操作。

14.1.1 模糊匹配算法

命令面板使用基于子序列的模糊匹配算法 (subsequence matching)，而非简单的子串匹配。算法流程：

1. **输入规范化**——将用户输入和所有候选项都转小写，去除非字母数字字符。
2. **子序列匹配**——检查用户输入的每个字符是否按顺序出现在候选项中。例如 `grpah` 对 `graph`：
`g→g✓, r→r✓, p→a×`, 跳过 `a`, `p→p✓, a→a✓, h→h✓` → 匹配成功。
3. **评分**——匹配成功后计算相关度分数：
 - 连续匹配字符加分 (`graph` 匹配 `graph` 比匹配 `generate paragraph` 分高)。
 - 匹配位置越靠前加分越多。
 - 完全匹配额外加分。
4. **排序**——按分数降序排列，取前 10 条结果。

这种算法的好处是容错性强——即使用户打错字 (`librray` → `library`)，只要字符按顺序出现就能匹配。

14.1.2 结果分组与展示

结果按类别分组展示：

- **Recent**——最近使用的 5 个命令和最近打开的 3 篇论文，固定显示在顶部（无需输入）。
- **Navigation**——匹配到的页面路由。
- **Papers**——匹配到的论文。
- **Actions**——匹配到的命令。

上下箭头移动选中项，`Enter` 执行，`Esc` 关闭。

提示 命令面板的设计目标是让所有功能都可以通过键盘到达。如果你不确定某个功能在哪里，在命令面板里搜一下通常比翻菜单更快。它也适合作为快速论文跳转工具——输入标题中的几个关键词就能找到目标论文。

自定义元数据字段

不同研究者有不同的组织方式。自定义元数据字段让你为论文添加任意键值对。

15.1 字段管理

在「设置 → 自定义字段」中定义字段。字段定义存储在 `custom_field_defs` 表中，字段值存储在 `paper_custom_fields` 表中。

15.1.1 字段类型

类型	存储格式	前端控件
text	任意字符串	文本输入框
number	数字字符串	数字输入框，支持排序比较
date	Unix 时间戳	日期选择器
select	选项值字符串	下拉选择框，选项在 <code>options</code> 字段中以 JSON 数组定义

15.1.2 数据模型

- `custom_field_defs`——字段定义表。每行一个字段：`name`（唯一）、`field_type`、`options`（`select` 类型的可选值 JSON 数组）、`created_at`。
- `paper_custom_fields`——字段值表。复合主键（`paper_id`，`field_id`），每行存储一个论文的一个字段值。

15.2 使用

字段定义后，会在所有论文的抽屉中出现一个「自定义字段」区域。你可以直接编辑每个论文的字段值。自定义字段支持：

- **筛选**——在主列表的筛选栏中,可以按自定义字段的值过滤论文。例如筛选 `project = thesis-ch3` 的所有论文。
- **排序**——`number` 和 `date` 类型的字段支持升序/降序排序。
- **导出**——在 Markdown 导出的 YAML frontmatter 中, 自定义字段以 `custom_< 字段名 >` 的格式写入。
- **全局定义**——创建一次, 所有论文共享相同的字段定义。删除字段定义会级联删除所有论文的该字段值。

提示 自定义字段与标签的区别: 标签是扁平的、无类型的关键词; 自定义字段是有类型的结构化数据 (如日期、数字、枚举)。如果你需要按「截止日期」排序或按「优先级」筛选, 用自定义字段比标签更合适。

/settings 路由是 LitFolio 的「管脚」。主要板块：**LLM 配置**、任务分配、**WebDAV** 同步、导出、自定义字段、标注颜色、主题提醒。

16.1 LLM 配置

LitFolio 不绑定任何一家 LLM 服务。它的设计是「OpenAI 兼容协议」：你可以加任意多个端点配置 (profile)，其中一个标记为「当前」。每个 profile 包含 API 地址、密钥、对话模型、向量模型 (可选)、采样温度。

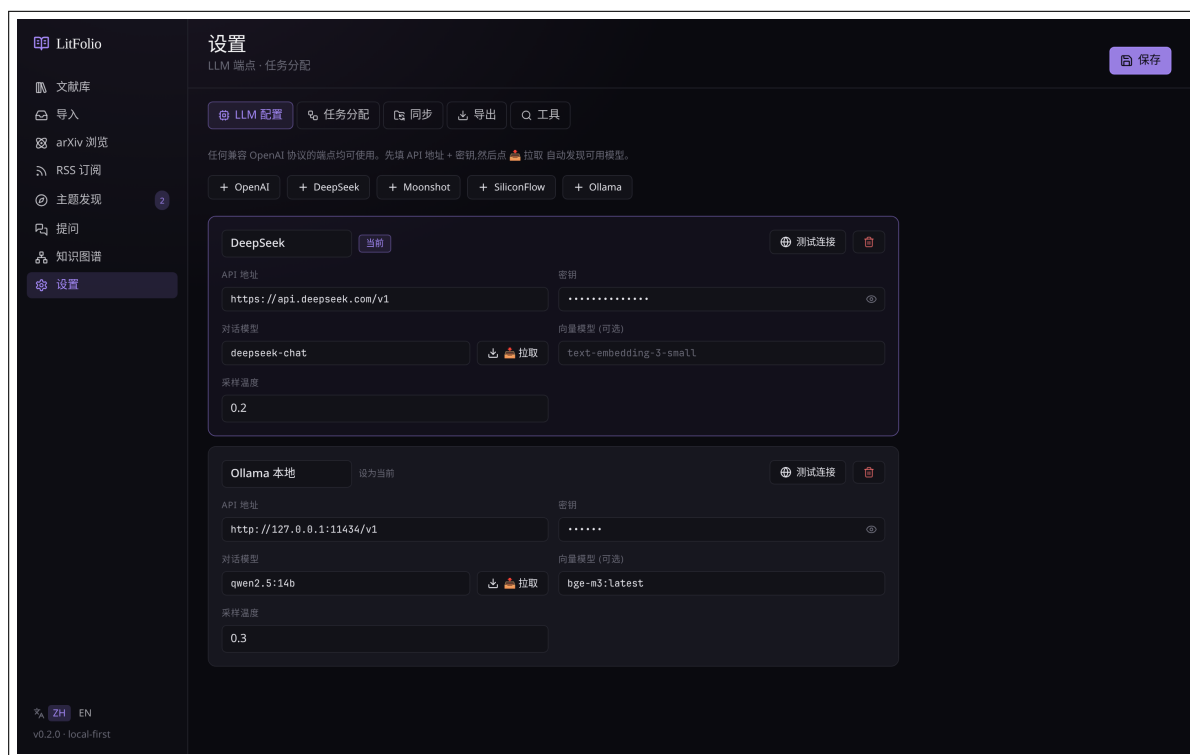


图 16.1 LLM 配置。两个 profile: DeepSeek 与本地 Ollama; 当前激活的是 DeepSeek。每个 profile 都能「📌 拉取」远端可用模型列表填进 chatModel 字段。

16.1.1 拉取可用模型

填好 API 地址与密钥后点  拉取，LitFolio 会调远端 GET /v1/models 拿可用模型列表。这一步可避免你手敲错 model name。返回的模型会填入下拉框。

16.1.2 测试连接

「测试连接」会发起一次小规模 chat（提示「你好」），用真实模型回复来验证整条链路。失败原因（鉴权、网络、模型不存在）会原样显示。

16.2 任务分配

LitFolio 把所有 LLM 调用归到 **6 个任务 (task)**：

- **速读 TL;DR**——一句话摘要 + 关键发现
- **深读 quickRead**——4 段式深度阅读
- **翻译 translate**——标题/摘要/选段翻译
- **搜索扩写 search**——主题搜索的查询扩写
- **综述 survey**——主题综述生成
- **术语 term**——术语表抽取
- **提问 ask**——库内 RAG 回答

每个任务可独立绑到任一 profile 与任一具体 model。所以你可以把：

贵但聪明 深读、综述、提问 → 选当代顶级闭源对话模型（如 OpenAI、Anthropic、Google 各家最新的旗舰），事关回答质量，但单次 token 量并不大。

便宜大碗 翻译、术语 → 选国内主流 API 服务的低价档（DeepSeek、Doubao、智谱、月之暗面等都有性价比极高的对话型号），翻译/术语调用量大，每百万 token 一两块以内就够用。

本地白嫖 TL;DR、搜索扩写 → 本地 **Ollama** 上的开源对话模型（Qwen、Llama、DeepSeek 蒸馏版等 7B~32B 任一），无成本、隐私好，速度看你显卡。

16.2.1 各任务的 LLM 调用细节

了解每个任务具体怎么调 LLM，有助于你判断该给它绑什么档次的模型。

速读 TL;DR

输入：标题 + 作者（前 6 位）+ 摘要。prompt 要求返回 JSON：{"tldr": "一句话，不超过 40 词", "key_findings": ["3-5 条，每条不超过 20 词"]}。输出语言跟随用户设置。JSON 解析有三级容错（直接解析 → 提取 {...} 子串 → 空对象兜底）。结果写入 papers 表缓存，再次点击不会重调 LLM。

深读 Quick Read

输入与 TL;DR 相同（标题 + 作者 + 摘要），但 prompt 更细致：要求返回四段结构化 JSON (problem / method / comparison / limitations)，每段 2–4 句散文体，不能用 bullet。system prompt 以「资深审稿人替同事判断该不该细读」的角色定位，引导 LLM 给出有判断力的评价，而非泛泛概括。

翻译

输入：标题 + 摘要。prompt 要求保留技术术语原文（模型名、算法名、数学符号、单位、数据集名）不翻译，不意译不概括，返回 JSON {"title": "...", "abstract": "..."}。

16.3 WebDAV 同步

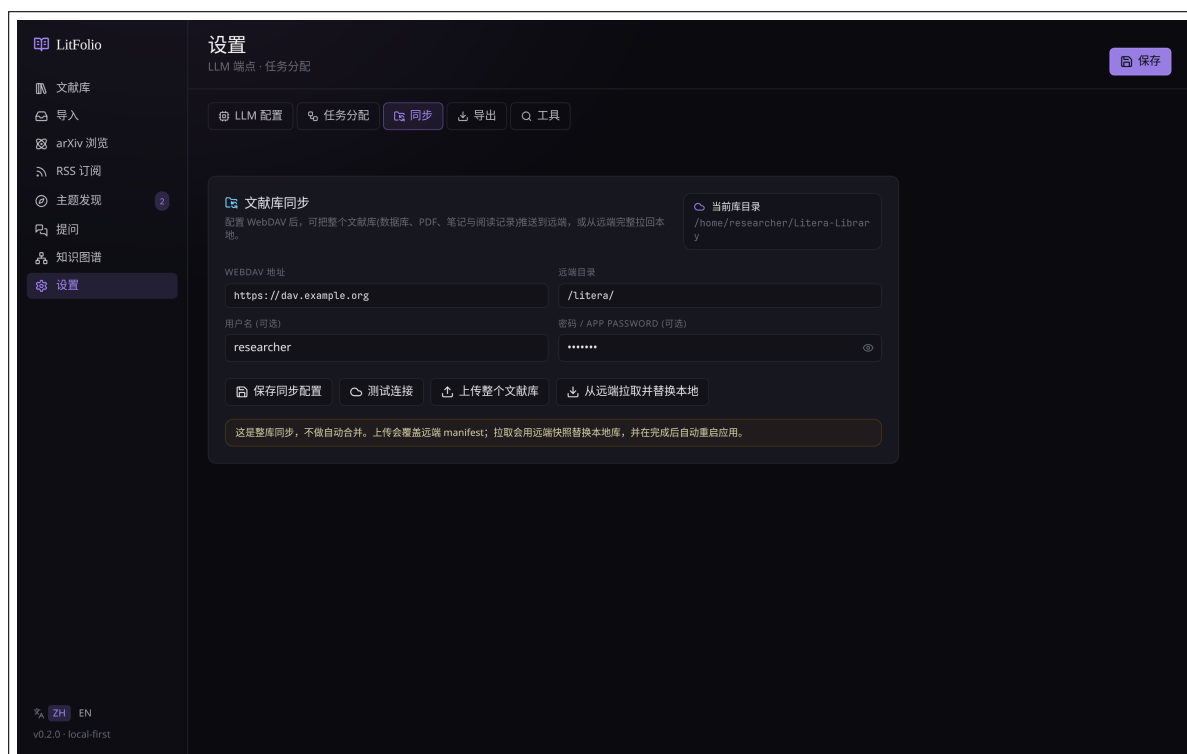


图 16.2 WebDAV 同步面板。配好 URL 与凭据后可**推送**（本地 → 远端）或**拉取并替换本地**（远端 → 本地，会先备份本地到 `backups/sync/`）。

LitFolio 的同步是「整库快照」语义，不是细粒度增量。具体来说：

1. **推送**——把整个 `~/Litera-Library/` 打包推到 WebDAV，远端会有完整副本。
2. **拉取并替换本地**——从远端拉一个完整副本，覆盖本地。

注意 「拉取并替换本地」会覆盖本地数据库与所有 papers/。LitFolio 在执行前会自动把当前本地状态保存到 backups/sync/<timestamp>/，所以即使你拉错方向也能回滚——但请认真对待这个操作，特别是当两台机器都有未推送变更时。

「整库覆盖」而不是增量合并的设计选择，是因为论文库的修改大多是 PDF 与笔记，冲突时哪一边胜出的语义对用户难以解释；只暴露「整体推/整体拉」与「自动备份」反而更可预测。

16.4 导出设置

「设置 → 导出」面板控制 Markdown 导出行为（详见第 14 章）：

- **导出目录**——选择 .md 文件的输出位置。
- **增量导出**——默认开启。关闭后每次导出全部论文。
- **自动导出**——开启后，笔记或高亮保存时自动触发导出（防抖 5 秒）。
- **立即导出**——手动触发一次完整或增量导出。

16.5 标注颜色设置

「设置 → 标注颜色」面板管理高亮颜色与语义标签的映射（详见 4.6 节）：

- 查看当前所有颜色-标签映射。
- 编辑标签名称。
- 添加新的颜色-标签映射。
- 删除不使用的映射。

16.6 自定义字段设置

「设置 → 自定义字段」面板管理论文的自定义元数据字段定义（详见第 16 章）：

- 创建新字段（名称 + 类型 + 可选的 select 选项）。
- 编辑已有字段。
- 删除字段（会级联删除所有论文的该字段值）。

16.7 主题提醒设置

「设置 → 主题提醒」面板管理自动化论文监控（详见 7.4 节）：

- 查看所有提醒及其上次运行时间。

- 查看每个提醒的历史结果列表。
- 编辑提醒的查询、频率、目标文件夹。
- 暂停或删除提醒。
- 标记结果为已读。

数据与文件布局

LitFolio 信奉「数据是用户的」。所有信息以可读格式落在 `~/Litera-Library/`，你不依赖 LitFolio 一样能用脚本访问。

17.1 顶层结构

<code>~/Litera-Library/</code>	
<code>library.db</code>	SQLite 主库
<code>library.db-wal</code>	WAL 日志（运行时）
<code>library.db-shm</code>	共享内存（运行时）
<code>papers/</code>	
<code><paperId>/</code>	
<code>paper.pdf</code>	
<code>meta.json</code>	
<code>note.md</code>	
<code>pdf-text.txt</code>	
<code>notes/</code>	
<code>ask/</code>	「保存为笔记」的 Ask 结果
<code>backups/</code>	
<code>deleted/</code>	删除前的 papers 副本
<code>sync/</code>	拉取前的本地快照
<code>sync/</code>	WebDAV 推送时使用的临时区

17.2 library.db 表概览

主要表（详细 schema 见仓库 [src-tauri/migrations/](https://github.com/src-tauri/migrations/)，共 24 个迁移文件）：

17.2.1 核心

- `papers`——核心元数据，每篇一行 (`id`、`title`、`authors`、`abstract`、`status`、`tldr`、`quick_read_json`、`translated_title`、`translated_abstract`、`bibtex`、`last_exported_at`...)
- `highlights`——选区高亮与区域高亮 (`paper_id`、`page`、`boundingRect`、`text`、`comment`、`color`、`label`)

- terms——术语表 (paper_id、term、definition、evidence、cross_refs)
- tags / paper_tags——标签与归属
- folders / paper_folders——文件夹与归属 (支持嵌套与多归属)

17.2.2 发现与阅读

- feeds / feed_items——RSS 源与条目
- topic_surveys——已保存的主题综述
- topic_alerts / topic_alert_results——主题提醒配置与结果
- reading_queue——阅读队列 (paper_id、priority、target_date、note)
- paper_comparisons——多论文对比结果
- recommendation_cache——相似论文推荐缓存
- paper_citations——引用网络缓存

17.2.3 笔记与知识

- paper_note_sections——结构化笔记卡片 (paper_id、section_key、content、source、sort_order)
- paper_links——论文间手动/AI 关系
- paper_terms——论文与术语的关联 (含跨论文复用)
- concepts——概念节点 (name、description、source)
- concept_relations——概念间关系 (replaces/extends/requires/enables/competes_with)
- paper_concepts——论文与概念的关联

17.2.4 配置与索引

- llm_profiles / task_assignments——端点与任务绑定
- papers_fts / highlights_fts / terms_fts——FTS5 全文索引
- paper_embeddings——向量嵌入缓存
- custom_field_defs / paper_custom_fields——自定义元数据
- smart_collections——智能收藏夹规则

17.3 papers/<id>/ 单篇布局

每篇论文一个目录，目录名是 ULID（01KSCN65XF4B2PZ83D27ET55PX 这种 26 字符）：

paper.pdf 原文 PDF。删除该文件 LitFolio 会显示「PDF 未绑定」，但元数据与笔记仍然在；用文献库抽屉里的「重新绑定 PDF」可挂回去。

meta.json 元数据快照（标题、作者、年份、DOI/arXiv ID、来源等）。SQLite 是主存储，meta.json 是供脚本读取的镜像。

note.md 这篇的笔记。完全由你写，LitFolio 不会改动除你在阅读器写入之外的内容。

pdf-text.txt PDF 全文提取（用 pdf-extract 离线生成），用于 FTS5 索引与 RAG 召回。

17.4 性能优化

LitFolio 针对大规模文献库（1000+ 篇论文）做了两项关键优化。

17.4.1 虚拟滚动

文献库主列表使用虚拟滚动（virtual scrolling）技术，基于 @tanstack/react-virtual 实现。核心原理：

1. **容器测量**——列表容器挂载时测量可视区域高度（如 800px）。
2. **行高估算**——每行论文卡片的估算高度（如 72px）。
3. **可视窗口计算**——根据滚动偏移量和容器高度，计算当前需要渲染的行索引范围。例如滚动到 500px 时，渲染第 7–18 行（加上上下各 3 行的 overscan 缓冲）。
4. **DOM 节点池**——只创建可视范围内的 DOM 节点。滚动时，离开可视区域的节点被回收，新进入的节点被创建。
5. **占位填充**——列表总高度通过 $totalSize * rowHeight$ 计算，用上下 padding 元素撑开滚动条，保证滚动行为与完整列表一致。

效果：

- 初始渲染时间 < 100ms（2000 篇论文）。
- 滚动保持 60fps——因为每次只操作 20–30 个 DOM 节点。
- 内存占用与可视区域大小成正比，而非与论文总数成正比。

17.4.2 嵌入缓存

RAG 查询（库内提问）需要对论文文本做向量嵌入。LitFolio 使用 paper_embeddings 表缓存已生成的嵌入向量：

1. 查询前，检查 `paper_embeddings` 表中是否有该论文的缓存记录，且 `content_hash` 与当前文本哈希一致。
2. 命中缓存 → 直接使用缓存的嵌入向量，跳过 API 调用。
3. 未命中 → 调用嵌入 API 生成向量，写入缓存表。
4. 论文内容变更（笔记编辑、高亮增删）→ `content_hash` 变化 → 下次查询时自动重新生成嵌入。

嵌入缓存对用户完全透明——不需要任何手动操作。日志中可以看到 "embedding cache hit" 或 "embedding cache miss" 来确认缓存是否生效。

17.5 备份

LitFolio 有两类自动备份：

1. **删除回收站** (`backups/deleted/`) —— **第 2 章** 讲过，删除论文会先移到这里。30 天后自动清理。
2. **同步快照** (`backups/sync/`) —— 「拉取并替换」前的本地完整快照。保留最近 5 次。

如果你需要自己加日定时备份,最简单的办法是把整个 `~/Litera-Library/` 作为 borg/restic 仓库源——SQLite WAL 文件也包括在内，可保证一致性。

快捷键速查

18.1 阅读器 (/reader)

快捷键	动作
Ctrl+F / Cmd+F	打开 PDF 搜索框
Enter	跳到下一个匹配
Shift+Enter	跳到上一个匹配
Esc	关闭搜索框
Alt+ 拖拽	区域高亮（矩形截图）
鼠标选择	文本高亮（弹气泡）
j / k	下一页 / 上一页
1 / 2 / 3	切换右栏笔记 / 选段翻译 / 术语
[/]	左/右栏宽度 5% 步进

18.2 全局

动作	效果
Ctrl+K / Cmd+K	打开命令面板
拖拽 PDF 到任意页面	跳到「导入 → PDF 文件」并预填该文件
Ctrl+Enter / Cmd+Enter	在 Ask 页发送消息
Esc	关闭命令面板 / 弹窗 / 搜索框

故障排查

下面把最常遇到的几类问题按「现象 → 原因 → 修复」整理。

19.1 LLM 调用失败

现象：TL;DR / 深读 / 翻译按钮转圈后弹「请求失败」。

常见原因与修复：

1. 当前 profile 没填或 model name 拼错——到「设置 → LLM 配置」点「测试连接」。
2. API key 余额耗尽 / 鉴权过期——错误详情会原样展示，参考服务商提示。
3. 网络不可达（特别是国内访问 OpenAI 直连）——换走兼容端点（Azure / 国内中转）。
4. 本地 Ollama 没起——ollama serve 拉起服务，再到设置点测试。

19.2 PDF 加载失败

现象：阅读器中间一直转圈或显示「加载 PDF 失败」。

修复：

- 检查 `papers/<id>/paper.pdf` 文件是否存在；不在则到文献库抽屉点「重新绑定 PDF」。
- 某些扫描版 PDF 用了非标准编码,PDF.js 渲染失败——可尝试用 `ocrmypdf` 或 `qpdf --linearize` 处理后重新绑定。

19.3 RSS 抓不到 / 一直「已是最新」

现象：明明远端有新内容，点「刷新」却显示「已是最新」。

原因：LitFolio 用条件 GET (If-Modified-Since / ETag)，远端说没变就不再拉。如果你怀疑远端在撒谎，可以从「订阅源管理」删除该源再重加。

User-Agent 被屏蔽：少数期刊网站会屏蔽默认 UA。在订阅时填入自定义 User-Agent 头即可。

19.4 WebDAV 同步失败

常见三种：

- **401 Unauthorized**——账号密码错；如用「Nextcloud」请用 **应用密码**而非账户密码。
- **404 / 405**——WebDAV URL 路径不对。Nextcloud 是 `/remote.php/dav/files/<user>/`，坚果云是 `https://dav.jianguoyun.com/dav/`。
- **冲突 / 中途失败**——拉取前的本地快照在 `backups/sync/`，可手动恢复。

19.5 数据库锁住

现象：启动时弹「database is locked」。

原因：意外 kill 后 WAL 文件残留，多数情况 SQLite 会自动 checkpoint 恢复。如果 LitFolio 一直起不来，关闭所有 LitFolio 进程后删 `library.db-wal` 与 `library.db-shm`（数据本身在 `library.db`，删 WAL 只丢未持久化的最近几秒变更）。

19.6 批量导入部分失败

现象：导入文件夹时，部分 PDF 显示「失败」。

常见原因：

1. **PDF 损坏**——文件不完整或被截断。用 `qpdf --check` 验证。
2. **无文本层**——扫描版 PDF 没有 OCR 文本层。LitFolio 仍然会入库（用文件名作为标题），但全文搜索无法覆盖内容。用 `ocrmypdf` 处理后重新绑定 PDF。
3. **权限问题**——PDF 文件被其他程序锁定或没有读权限。

19.7 概念提取结果为空

现象：点击「提取概念」后，知识图谱中没有出现概念节点。

常见原因：

1. 论文没有 TL;DR 或深读结果——LLM 缺乏足够上下文来识别概念。先对论文运行速读或深读。
2. LLM 返回了无法解析的 JSON——检查 LLM 配置的模型是否支持 JSON 输出格式。
3. 论文内容太短或不是学术论文——概念提取对学术论文的效果最好。

19.8 引用网络/相似论文为空

现象：点击「引用」或「相似论文」后显示「暂无数据」。

原因：

- 论文没有 DOI 且标题匹配不到 Semantic Scholar 中的记录。
- 论文非常新（发表不到 1 周），S2 尚未收录。
- 小众领域或非英文论文，S2 覆盖率有限。
- 网络问题——检查是否能访问 `api.semanticscholar.org`。

19.9 智能收藏夹不更新

现象：修改了论文的标签或状态，但智能收藏夹的论文列表没有变化。

原因：智能收藏夹是实时查询的——你需要重新选中该收藏夹才能看到更新。这不是 bug，而是设计选择：避免每次论文变更都触发规则查询影响性能。

LLM 端点兼容清单

LitFolio 遵循「OpenAI Chat Completions 协议」，下面这些端点都验证过可直接接入：

服务	API 地址	备注
OpenAI 官方	https://api.openai.com/v1	直连可能受限
DeepSeek	https://api.deepseek.com/v1	推荐：性价比高
Azure OpenAI	<resource>.openai.azure.com/ openai/deployments/<dep>/v1	需要 API 版本头
Ollama（本地）	http://localhost:11434/v1	完全离线
LM Studio（本地）	http://localhost:1234/v1	桌面 GUI 加载 GGUF
OpenRouter	https://openrouter.ai/api/v1	一个 key 多模型
Together.ai	https://api.together.xyz/v1	开源模型托管

任务分配推荐思路（具体型号迭代很快，按思路选当下的就行）：

- **学生 / 全免费**——能本地的全本地（本地 Ollama 跑 7~14B 开源模型已够用于 TL;DR / 翻译 / 搜索扩写）；非要联网的留给「深读」「综述」「提问」，挑国内厂商的免费层即可。
- **研究者 / 性价比**——所有任务统一指向**国内主流 API 的低价档**（DeepSeek、Doubao、智谱、月之暗面等任选一家），单价低、上下文窗口大，绝大多数研究场景已经够用。
- **硬核 / 不计成本**——TL;DR / 翻译 / 术语丢国内便宜档；深读 / 综述 / 提问交给**当时最强的旗舰对话模型**（OpenAI、Anthropic、Google 各家轮流领先，按发布时间挑一个即可），保证回答上限。

arXiv 主流分类速查

cs.AI / cs.LG / cs.CL 人工智能 / 机器学习 / 自然语言处理

cs.CV 计算机视觉

cs.CR / cs.DC 密码与安全 / 分布式系统

stat.ML 统计机器学习

physics.optics 光学与光子学

physics.atom-ph 原子物理

cond-mat.mes-hall 凝聚态-介观与纳米

quant-ph 量子物理

math.OC 优化与控制

math.AP 偏微分方程

q-bio.NC 神经科学

完整列表见 https://arxiv.org/category_taxonomy。

推荐 RSS 源

arXiv 子分类（替换 CAT 为如 cs.LG）：

- <http://export.arxiv.org/rss/CAT>——官方 RSS，每日刷新
- <http://export.arxiv.org/rss/CAT.recent>——只看最新

期刊：

- Nature: <https://www.nature.com/nature.rss>
- Science: https://www.science.org/rss/news_current.xml
- Nature Photonics: <https://www.nature.com/nphoton.rss>
- Optica (OSA): <https://www.optica-opn.org/rss/news.xml>
- PRL: <http://feeds.aps.org/rss/recent/prl.xml>

研究博客 / 综述：

- Distill: <https://distill.pub/rss.xml>（已停更，但旧文价值高）
- Lil'Log: <https://lilianweng.github.io/index.xml>

许可证

LitFolio 使用 **GPL-3.0-or-later** 许可证发布。这意味着：

- 你可以自由使用、修改、再分发。
- 如果你基于 LitFolio 做了衍生作品并对外分发，衍生作品也必须以同样许可证（或更宽松的兼容许可证）发布，并提供源码。
- 不提供任何形式的明示或暗示担保。

完整许可证文本见仓库根目录 [LICENSE](#) 文件。